

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ»

ЯЗЫК ПРОГРАММИРОВАНИЯ PYTHON

Часть 1

Учебно-методическое пособие

Воронеж
Издательский дом ВГУ
2021

Утверждено научно-методическим советом факультета прикладной математики, информатики и механики 15 февраля 2021 г., протокол № 6

Составители: М. А. Артемов, Е. С. Барановский, Р. Г. Меджидов

Рецензент – кандидат физико-математических наук, доцент кафедры вычислительной математики и прикладных информационных технологий ВГУ
К. П. Лазарев

Учебно-методическое пособие подготовлено на кафедре программного обеспечения и администрирования информационных систем факультета прикладной математики, информатики и механики Воронежского государственного университета.

Рекомендовано для бакалавров 1-го курса факультета прикладной математики, информатики и механики Воронежского государственного университета.

Для направлений: 02.03.03 – Математическое обеспечение и администрирование информационных систем, 09.03.03 – Прикладная информатика

Содержание

1.	Обзор языка Python.....	6
1.1.	Применимость языка Python.....	6
1.2.	Выполнение программы.....	7
2.	Особенности языка Python.....	9
2.1.	СИ-подобные языки.....	9
2.2.	Особенности синтаксиса	10
2.3.	Динамическая типизация	12
2.4.	Сборка мусора	15
2.5.	Подсказки.....	16
3.	Типы данных	18
3.1.	Основные типы данных.....	18
3.2.	Числовые типы	19
3.3.	Строки	23
3.4.	Булевские значения.....	27
3.5.	Неизменяемость.	27
3.6.	Переменные в Python.....	29
4.	Операторы и выражения	30
4.1.	Выражения.....	30
4.2.	Порядок выполнения операций.....	32
4.3.	Преобразование типов в выражения.....	33
4.4.	Операции сравнения	34
4.5.	Операции деления.....	35
5.	Задачи	36
5.1.	Пример решения.....	36
5.2.	Арифметические задачи.....	39
5.3.	Задачи на строки	40
6.	Консольный ввод-вывод.....	41

6.1.	Управляющие последовательности	41
6.2.	Преобразования.....	43
6.3.	Форматирование строк	44
6.4.	Функции print и input	49
7.	Ветвления и циклы.....	50
7.1.	Ветвления.....	50
7.2.	Логические операторы	51
7.3.	Трехместное выражение if else	52
7.4.	Циклы	52
8.	Последовательности и отображения.....	54
8.1.	Списки.....	54
8.2.	Словари	58
8.3.	Кортежи.....	60
9.	Задачи.....	61
9.1.	Пример решения.....	61
9.2.	Задачи на форматированный вывод.....	64
9.3.	Задачи на списки	66
10.	Модульность.....	67
10.1.	Модули	67
10.2.	Конфликты имен в модулях.....	70
11.	Функции	70
11.1.	Основные понятия	70
11.2.	Область видимости	74
12.	Прочие основные типы.....	76
12.1.	Файлы.....	76
12.2.	Множества	79
12.3.	None	80
12.4.	Объект type	81

13.	Задачи.....	81
13.1.	Пример решения.....	81
13.2.	Список задач.....	90

1. Обзор языка Python

1.1. Применимость языка Python

Python — это скриптовый язык программирования. Он универсален, и поэтому подходит для решения разнообразных задач и многих платформ, начиная с iOS и Android и заканчивая серверными ОС.

Это интерпретируемый язык — он не компилируется, т.е. до запуска представляет собой обычный текстовый файл. Программировать можно практически на всех платформах, язык хорошо спроектирован и логичен.

Чаще всего Python используется в веб-разработке. Для работы с ним подключают *фреймворки*: Pyramid, Pylons, TurboGears, Flask, CherryPy и самый популярный — Django.

Несмотря на то, что язык не компилируется, с его помощью создают десктопные программы, например:

- GIMP — визуальный редактор на Linux;
- Ubuntu Software Center — центр приложений в ОС Ubuntu (один из дистрибутивов Linux);
- BitTorrent до 6 версии — менеджер торрент-закачек (позже программу переписали на C++, но сети peer-to-peer всё ещё работают на Python);
- Blender — программа для создания 3D-графики.

Мобильная разработка на Python менее популярна. Для Android чаще всего используются Java, C#, C++ или Kotlin, а для iOS — Swift или Objective-C. На Python обычно создается серверная часть приложения. Например, клиент Instagram для iOS написан на Objective-C, а сервер — на Python.

Многие компьютерные игры полностью или частично написаны на Python. Несмотря на возможность реализации пользовательского

интерфейса и работы с графикой, на Python в основном пишут скрипты — такие как взаимодействия персонажей, запуск сцен, а также обработка событий.

На Python разрабатываются встроенные системы для различных устройств. Например, язык используется в Raspberry Pi (компьютер размером с карту памяти) и в «Сбербанке» для управления банкоматами.

Язык применяется во встроенных системах станков с ЧПУ, средствах автоматического регулирования (температуры, расхода жидкостей, давления и т. д.) и в телекоммуникационном оборудовании.

Язык Python очень популярен и входит в топ-3 самых востребованных языков программирования ежегодно, начиная с 2016 года.

1.2. Выполнение программы

Python представляет собой программный пакет, называемый *интерпретатором*. Интерпретатор — это разновидность программы, которая выполняет другие программы. Он читает написанную программу и приводит в исполнение содержащиеся в ней инструкции. Фактически интерпретатор является уровнем программной логики между вашим кодом и оборудованием компьютера. В зависимости от используемой разновидности Python сам интерпретатор может быть реализован в виде программы C, набора классов Java или чего-либо еще.

В своей простейшей форме программа на Python представляет собой текстовый файл, содержащий операторы. Можно создать такой файл с операторами в любом текстовом редакторе. По соглашению, файлам с программами Python назначаются имена, оканчивающиеся на *.py*.

После набора всех операторов в текстовом файле, необходимо сообщить интерпретатору Python о необходимости выполнить файл (через строки командной оболочки или с помощью IDE-сред).

После запуска Python внутренне *компилирует* исходный код в *байт-код*. Компиляция является просто шагом трансляции, а байт-код — это низкоуровневое и независимое от платформы представление исходного кода. Другими словами, Python транслирует каждый оператор исходного кода в группу инструкций байт-кода, разбивая их на отдельные шаги. Байт-код способен выполняться быстрее, чем первоначальные операторы исходного кода из текстового файла.

Если процесс Python имеет доступ по записи на данном компьютере, тогда он будет сохранять программы в файлах с расширением *.рус*. Во время следующего запуска программы, Python загрузит такой файл и пропустит шаг компиляции при условии, что исходный код не был изменен после того, как байт-код был сохранен, и не используется другая версия Python.

Если Python не имеет возможности записывать файлы байт-кода на устройстве, то программа все равно будет работать — байт-код генерируется в памяти и отбрасывается по завершении выполнения программы.

После того, как программа скомпилирована в байт-код, она отправляется на выполнение *виртуальной машине* Python (Python Virtual Machine — PVM). Машина PVM не является отдельной программой и сама по себе не требует установки. По сути, машина PVM — всего лишь большой закодированный цикл, который проходит по инструкциям байт-кода, чтобы выполнить их операции. PVM — это исполняющий механизм Python; она всегда присутствует в виде части системы Python и представляет собой компонент, который по-настоящему выполняет ваши сценарии. Формально она является просто последним шагом того, что называется интерпретатор Python.

Вся эта сложность намеренно скрыта от программистов на Python. Компиляция в байт-код происходит автоматически, а машина PVM — лишь часть системы Python, установленной на компьютере.

Определенный код Python может выполняться не настолько быстро, как код C или C++. Во-первых, в работе Python обычно отсутствует шаг построения или *сборки*: код выполняется немедленно после того, как был набран. Во-вторых, байт-код Python не является двоичным машинным кодом — это представление, специфичное для Python.

С другой стороны, в отличие от классических интерпретаторов по-прежнему имеется внутренний шаг компиляции — Python не нуждается в частом повторном анализе и разборе текста каждого оператора исходного кода. В результате чистый код Python выполняется со скоростью, находящейся между скоростью традиционного компилируемого языка и скоростью традиционного интерпретируемого языка.

С фундаментальной точки зрения в Python имеется лишь исполняющий механизм — начальная стадия компиляции отсутствует и все делается во время выполнения программы. Сюда входят даже такие операции, как создание функций и классов, плюс связывание модулей. В статических языках указанные события происходят перед стадией выполнения, в то время как в Python — во время выполнения.

2. Особенности языка Python

2.1. СИ-подобные языки

К СИ-подобным языкам часто относят языки C++, D, Java, Objective C, C#, PHP, perl, Nemerle, scala, Python и прочие.

Си-подобный синтаксис характеризуется следующими особенностями:

1. Чувствительность к регистру.

2. В языке есть встроенная поддержка числовых, символьных (обрамляются одинарными кавычками) и строковых (обрамляются двойными кавычками) литералов.

3. Для представления специальных символов в символьных и строковых литералах используется обратная косая черта.

4. Двухместные операторы имеют инфиксную форму.

5. Для группировки выражений используются круглые скобки.

6. Для записи вызова функции используется имя функции, за которым следует заключённый в круглые скобки список аргументов, разделённых запятыми.

7. Для обращения к элементу массива или индексатору используются квадратные скобки.

8. В языке есть предопределённый набор ключевых слов, которые состоят из английских букв в нижнем регистре (и, возможно, знаков подчеркивания).

Эти критерии справедливы для языка Python. Однако следующие пункты в языке не выполнены или выполнены частично:

9. Количество пробелов, табуляция (отступы) и переводы строки не влияют на семантику нигде, кроме строковых литералов. Это, в частности, значит, что возможен перевод строки в любом удобном месте.

10. Для разделения инструкций используется точка с запятой.

11. Разрешены однострочные и многострочные комментарии. Однострочные комментарии обозначаются видом «//», а многострочные видом «/*.....*/».

12. Для отделения блоков инструкций используются фигурные скобки.

2.2. Особенности синтаксиса

Первая из ярких особенностей синтаксиса языка — это конец оператора (инструкции). В языке Python конец строки является концом

оператора, то есть символ «точка с запятой» не требуется. Наличие точки с запятой в конце строки не будет являться ошибкой, однако нарушает стиль программирования Python.

Вторая заметная особенность — в коде нет явно набираемых программистом синтаксических пометок начала и конца вложенного блока кода. То есть блок не помещается внутрь `begin/end` или фигурных скобок.

Вложенные инструкции объединяются в блоки по величине отступов. Отступ можно задавать любым, но одинаковым в пределах одного вложенного блока. В противном случае интерпретатор решит, что это различные блоки.

Третья особенность — символ двоеточия. Все составные операторы в Python следуют общему шаблону:

строка_заголовка: вложенный_блок_кода

Хотя операторы обычно располагают по одному в строке, в Python разрешено помещать несколько операторов в одну строку, разделяя их точками с запятой:

```
x = 0; y = 0
```

Такое применение допускается, только если объединяемые подобным образом операторы сами не являются составными.

Допустимо разнести одиночный оператор на множество строк. Для этого понадобится лишь поместить часть оператора внутри пары скобок — круглых `()`, квадратных `[]` или фигурных `{}`. Альтернативный вариант — добавить в конец предшествующей строки обратную косую черту, чтобы указать на продолжение оператора в следующей строке.

```
if (x == 0 and
    y == 0):
    print('0 0')
```

Тело вложенной инструкции может располагаться в той же строке, что и тело основной, если оно достаточно компактно:

```
if x == 1: print('one')
```

В коде Python текст, находящийся после символа `#` и продолжающийся до конца строки, считается *комментарием* и игнорируется Python. Комментарии — это способ написания документации по коду, предназначенной для чтения человеком, и важная часть программирования.

Комментарии всегда должны быть короткими, прямолинейными и добавлять информативную ценность.

```
x = 5      # присваиваем пятерку иксу
```

Это строчный комментарий, не несущий полезной информации.

```
tax = 1.1   # пошлина 10%
```

Это более полезный комментарий, поясняющий код.

2.3. Динамическая типизация

Программисты Python не объявляют конкретные типы объектов, используемые в сценариях. Большинство программ даже не должны заботиться о конкретных типах; взамен иногда они становятся естественным образом применимыми в большем числе контекстов, чем запланировано.

Динамическая типизация является с одной стороны корнем гибкости языка, но с другой — потенциальным камнем преткновения.

Типы в Python определяются автоматически во время выполнения, а не в ответ на объявления в коде. Поэтому переменные никогда не объявляются заблаговременно.

Переменная (также называемая в Python *именем*) создается при первом присваивании ей значения в коде. Последующие присваивания изменяют значение уже созданного имени.

Переменная никогда не располагает какой-либо информацией о типе или о связанных с ним ограничениях. Понятие типа обитает в объектах, не в именах. Переменные являются обобщенными по своей природе; они всегда просто ссылаются на определенный объект в конкретный момент времени.

Когда переменная встречается в выражении, она тотчас же заменяется объектом, на который в текущий момент ссылается, каким бы он ни был. Более того, все переменные должны быть присвоены, прежде чем их можно будет применять; ссылка на неприсвоенные переменные приводит к ошибкам.

Такая модель динамической типизации разительно отличается от модели типизации традиционных языков.

Какие на самом деле процессы протекают при присвоении значений, показано на примере:

`d = 5`

1. Создание объекта для представления значения 5.
2. Создание переменной `d`, если она еще не существует.
3. Связывание переменной `d` с новым объектом 5.

Переменные и объекты хранятся в разных местах памяти и связываются посредством ссылок. Переменные всегда ссылаются на объекты и никогда на другие переменные.

Такие связи от переменных к объектам в Python называются ссылками. Всякий раз, когда переменная используется в более позднее время, Python автоматически следует по связи от переменной к объекту.

Переменные — это записи в системной таблице, в которых предусмотрены места для связей с объектами. *Объекты* — это области выделенной памяти с достаточным пространством для представления

значений, для которых они предназначены. *Ссылки* — это следуемые указатели от переменных к объектам.

Типы имеют объекты, а не переменные. Это можно увидеть из примера:

```
g = 5
```

```
g = 'five'
```

```
g = 5.001
```

Переменная *g* начинает свое существование как целое число, затем становится строкой и, наконец, числом с плавающей точкой.

Это может показаться странным программистам на языках со статической типизацией, поскольку выглядит так, будто бы тип *g* изменяется с целочисленного на строковый, когда вводится *g='five'*. На самом же деле, *имена не имеют типов*, типы обитают в объектах. В действительности изменяется не тип переменной *g*, а *переменная становится ссылкой на объект другого типа*.

Следующий пример показывает, что в Python называется разделяемым объектом или разделяемой ссылкой:

```
q = 22
```

```
r = 22
```

Переменные *q* и *r* ссылаются на тот же самый объект, то есть указывают на один и тот же участок памяти.

Имена *q* и *r* не связаны друг с другом напрямую. Следовательно, следующая строчка кода не изменит значение в переменной *r*:

```
q = 1520
```

Это также означает, что в результате следующего кода, переменные *t* и *u* будут равны 0 и 5 соответственно:

```
t = 5    # t = 5
```

```
u = t    # u = 5
```

```
t = 0    # t = 0, но u = 5
```

Этот эффект не касается изменяемых объектов, таких как списки:

```
listA = [1, 2]    # Создается изменяемый объект
```

```
listB = listA     # Ссылка на тот же объект
```

```
listA[0] = -1     # Изменение на месте
```

В итоге таких действий обе переменные (listA и listB) указывают на один и тот же список, который изменится с [1, 2] на [-1, 2]. Следующие строки

```
print(listA)
```

```
print(listB)
```

выведут одно и то же значение [-1, 2].

Такое поведение характерно только для изменяемых объектов, которые поддерживают изменения на месте.

Проверить два объекта на эквивалентность (равенство значений) можно с помощью оператора проверки на равенство ==, например:

```
x == y
```

Проверить являются ли две переменные одним и тем же объектом можно с помощью оператора is:

```
x is y
```

2.4. Сборка мусора

В Python, при присваивании имени нового объекта, область памяти, занимаемая предыдущим объектом, освобождается в случае, если на него не ссылается любое другое имя или объект. Такое автоматическое

освобождение памяти объектов известно как *сборка мусора*, и она значительно упрощает жизнь программистов на языках, которые ее поддерживают.

Например:

```
h = 5
```

```
h = 12 # Освободить память, занимаемую 5
```

```
h = 'hi' # Освободить память, занимаемую 12
```

Память будет освобождена, только если на этот же объект нет другой ссылки.

Внутренне Python совершает такой ловкий трюк за счёт ведения в каждом объекте счетчика, который отслеживает количество ссылок, в текущий момент указывающих на объект. Как только и точно когда этот счетчик уменьшается до нуля, область памяти объекта автоматически освобождается.

2.5. Подсказки

Встроенная функция *dir* предоставляет легкий способ получения перечня всех атрибутов, доступных внутри объекта (то есть его методов и более простых элементов данных). Ее можно вызывать без аргументов, чтобы получить список переменных в области видимости вызывающего кода. Но более удобно то, что *dir* можно вызывать на любом объекте, имеющем атрибуты. Такие объекты могут быть в том числе импортированными модулями и встроенными типами, а также именами типов данных.

Например, для выяснения, что доступно в модуле, подобном *sys* из стандартной библиотеки, его следует импортировать и передать *dir*:

```
import sys
```

```
dir(sys)
```

Чтобы выяснить, какие атрибуты предоставляют объекты встроенных типов, можно запустить *dir* с литералом или существующим экземпляром желаемого типа. Например, для просмотра атрибутов списка и строки можно передать пустые объекты:

```
dir([])
```

```
dir('')
```

Получать перечни атрибутов встроенных типов можно и за счет передачи функции *dir* имени типа вместо литерала.

Помимо комментариев #, в Python поддерживается документация, которая автоматически присоединяется к объектам и сохраняется во время выполнения для инспектирования. Синтаксически, такие комментарии записываются как строки в начале файлов модулей, а также операторов функций и классов, перед любым другим исполняемым кодом. Python автоматически помещает текст таких строк, известных как *строки документации*, в атрибуты `__doc__` соответствующих объектов.

Например, в модуле `mod_1` создана и задокументирована функция:

```
def sqr(x):  
    ...  
  
    Берем x и возводим в квадрат  
    ...  
  
    return x ** 2
```

Теперь, при импортировании модуля, в атрибутах `__doc__` можно увидеть комментарии к функции:

```
import mod_1

print(mod_1.sqr.__doc__)
```

Встроенные модули и объекты в Python также используют строки документации.

3. Типы данных

3.1. Основные типы данных

В таблице приведен обзор встроенных типов объектов Python и примеров синтаксиса, используемого для кодирования их *литералов*, т. е. выражений, которые генерируют такие объекты.

Таблица: обзор встроенных типов объектов	
Тип объекта	Пример литерала/создания
Числа	123, 3.1415, 2+7j, 0b10011
Строки	'text', "Large text"
Списки	[1, 2, 3], [1, "two", [3, 4]]
Словари	{53: 'Jason', 73: 'Arnold'}
Кортежи	(1, 2, "three", 'A')
Файлы	open('myfile.txt')
Множества	set('hello'), {'a', 'b', 'c', 'd'}
Прочие основные типы	Булевские значения, сами типы, None
Типы программных единиц	Функции, модули, классы
Типы, связанные с реализацией	Скомпилированный код, трассировки стека

Типы объектов из таблицы достаточно универсальные и мощные. Списки предоставляют упорядоченные коллекции других объектов, а словари хранят объекты по ключам; и списки, и словари могут быть вложенными, расти и уменьшаться по требованию и содержать объекты любого типа.

Программные единицы — функции, модули и классы — в Python также представляют собой объекты. Они могут создаваться и могут свободно передаваться между сценариями, храниться внутри других объектов и т. д. Также Python предлагает набор типов, связанных с реализацией, таких как объекты скомпилированного кода.

Представленную таблицу в действительности нельзя считать полной, поскольку все, что обрабатывается в программах Python, является какой-либо разновидностью объекта. Тем не менее, данные объекты обычно называются основными типами данных, потому что они фактически встроены в язык Python, т. е. для генерации большинства из них имеется специфический синтаксис выражений.

Необходимо еще раз отметить, что после того, как объект создан, он навсегда привязывается к своему набору операций. Это и означает, что язык Python динамически типизирован — использует модель, которая отслеживает типы автоматически, не требуя объявления, но он также строго типизирован — ограничение, обуславливающее возможность выполнения на объекте только допустимых для его типа операций.

3.2. Числовые типы

Данное понятие включает в себя: *целые числа*, не имеющие дробной части, *числа с плавающей точкой*, которые имеют дробную часть, и более экзотические типы — *комплексные* числа с мнимой частью, *десятичные* числа с фиксированной точностью, *рациональные* числа с числителем и знаменателем, *булевы* числа, а также *полнофункциональные множества*.

Числа в Python поддерживают обычные математические операции. Например, плюс (+) выполняет сложение, звездочка (*) используется для умножения, а две звездочки (**) применяются для возведения в степень.

```
A = 2
B = 3
k = A + B # 5
m = A - B # -1
n = A * B # 6
p = A / B # 0.6666666666666666
q = A ** B # 8
```

Python при необходимости автоматически обеспечивает повышенную точность для больших чисел. Например, результатом операции `2**100` будет число 1267650600228229401496703205376.

Помимо выражений в состав Python входит несколько полезных числовых модулей для работы с числами. Например, модуль `math` содержит более сложные числовые инструменты в виде функций, а модуль `random` выполняет генерацию случайных чисел и случайный выбор.

Python позволяет записывать целые числа с использованием шестнадцатеричных, восьмеричных и двоичных литералов:

```
on = 0o1750# 1000 в восьмеричной системе
xn = 0x3e8 # в шестнадцатеричной
bn = 0b1111101000 # и в двоичной
```

Литералы шестнадцатеричных чисел начинаются с символов «0x» или «0X», за которыми следует строка шестнадцатеричных цифр (0-9 и A-F). Шестнадцатеричные цифры могут записываться в нижнем или верхнем регистре.

Литералы восьмеричных чисел начинаются с символов «0o» или «0O» (ноль и буква «o» в нижнем или верхнем регистре), за которыми следует строка восьмеричных цифр (0-7).

Литералы двоичных чисел начинаются с символов «0b» или «0B», за которыми следуют двоичные цифры (0-1).

Если попробовать напечатать числа из примера, то результатом будет число 1000 в десятичной системе. Чтобы представить число в двоичной, восьмеричной или шестнадцатеричной системе, существуют функции соответственно *bin*, *oct* и *hex*. Например:

```
print(hex(88005553535))
```

Данная строчка кода выведет это число в шестнадцатеричном виде: «0x147d8a2d7f».

Комплексные числа в Python записываются как

действительная_часть+мнимая_часть,

где мнимая часть заканчивается символом «j» или «J». Формально, действительная часть необязательна, так что мнимая часть может появляться сама по себе. Внутренне комплексные числа реализованы в виде пары чисел с плавающей точкой, но все числовые операции выполняют математику комплексных чисел, когда применяются к таким числам. Комплексные числа можно также создавать вызовом встроенной функции *complex(real,imag)*.

Пример создания комплексных чисел:

```
c1 = 3 + 2j
```

```
c2 = complex(3, 2)
```

Десятичные числа (тип *Decimal*) подобны числам с плавающей точкой, но имеют фиксированное количество разрядов после десятичной точки. Следовательно, десятичные числа являются значениями с плавающей точкой фиксированной точности.

```
from decimal import Decimal
```

```
x = Decimal('0.1')
```

Например, благодаря десятичным числам можно иметь значение с плавающей точкой, которое всегда удерживает всего лишь два десятичных разряда после точки. Кроме того, можно указывать, каким образом округлять или усекать лишние десятичные разряды за пределами границ объекта. Хотя десятичный тип в общем случае приводит к снижению производительности, если сравнивать его с нормальным типом с плавающей точкой, он хорошо подходит для представления величин с фиксированной точностью вроде денежных сумм и может помочь в достижении лучшей числовой точности.

Лучшая точность означает, например, отсутствие погрешности в случае использования десятичных типов:

```
from decimal import Decimal

x = Decimal('0.1')

y = 0.1

print(x + x + x)    # 0.3

print(y + y + y)    # 0.30000000000000004
```

Еще один числовой тип — это дробный тип `Fraction`, который реализует объект рационального числа. По существу он явно хранит числитель и знаменатель, так что избегает ряда неточностей и ограничений математики с плавающей точкой.

```
from fractions import Fraction

fr1 = Fraction(1, 3)

fr2 = Fraction(4, 6)
```

Переменная `fr1` хранит 1 в числителе и 3 в знаменателе, переменная `fr2` — упрощается по общему делителю до $2/3$.

Данный тип также поддерживает обычные операции над числами:

```
from fractions import Fraction

x = Fraction(1, 2) # 1/2

y = Fraction(1, 3) # 1/3

print(x + y)      # 5/6
```

3.3. Строки

Строка — упорядоченная коллекция символов, используемая для хранения и представления текстовой и байтовой информации.

Строки являются первым примером того, что в Python называется последовательностью — позиционно упорядоченной коллекцией других объектов. Для содержащихся элементов последовательности поддерживают порядок слева направо: элементы сохраняются и извлекаются по своим относительным позициям.

Как последовательности, строки поддерживают операции, которые предполагают наличие позиционного порядка среди элементов. Например, можно проверить длину строки с помощью встроенной функции `len` или извлечь ее компоненты посредством выражений индексации:

```
st = 'text' # Создание строки
```

```
len(st)      # Длина строки
```

Результатом выполнения будет число 4.

```
st[0]        # Первый элемент строки
```

Результатом окажется символ «t», так как индексация в языке начинается с нулевой позиции.

Индексы в Python представляют собой смещения спереди и поэтому начинаются с 0: первый элемент находится по индексу 0, второй — по индексу 1 и т.д.

В Python можно также индексировать в обратном направлении, с конца, т.е. положительные индексы считаются слева, а отрицательные — справа:

```
st[-2]      # Второй элемент с конца
```

Если вывести значение, полученное в примере, то это будет второй с конца элемент, т. е. «х».

В квадратных скобках можно применять произвольное выражение, а не только жестко закодированный числовой литерал — везде, где Python ожидает значение, допускается использовать литерал, переменную или любое желаемое выражение. Например, `st[5-3]` обращается к тому же элементу, что и `st[2]`.

В дополнение к простой позиционной индексации последовательности также поддерживают более общую форму индексации, известную как *нарезание*, которое представляет собой способ извлечения целой части (среза) за один шаг.

```
st[1:3]
```

Значение такого среза – строка «ех».

Срез `X[i:j]` означает «предоставить из `X` все содержимое, начиная со смещения `i` и заканчивая смещением `j`, не включая его». Результат возвращается в новом объекте.

Одно или оба из значений в квадратных скобках может быть опущено. Если отсутствует первое значение — срез начинается с начала строки, если последнее — срез будет до конца строки:

```
s = '0123456789'
x = s[:5]   # '01234'
y = s[5:]   # '56789'
z = s[:]    # '0123456789'
```

Расширенное нарезание `X[i:j:k]` принимает шаг (страйд) `k`, который по умолчанию равен `+1`:

```
A = 'персиквжести'
B = A[::2] # 'привет'
```

Можно также использовать отрицательный страйд для сохранения элементов в противоположном порядке:

```
A = 'А роза упала на лапу Азора'
B = A[::-1]
# B = 'арозА упал ан алапу азор А'
```

Строки поддерживают также *конкатенацию* (объединение двух строк в новую строку), обозначаемую знаком плюс, и *повторение* (создание новой строки путем повторения другой):

```
'hell' + 'o'
'Woo-Woo ' * 3
```

Результат первой строки кода: слияние двух строк «hello», результат второй — трехкратное повторение строки, то есть «Woo-Woo Woo-Woo Woo-Woo».

Python выполняет автоматическую конкатенацию соседних строк в любом выражении, хотя почти также просто поместить между ними операцию `+`, чтобы инициировать конкатенацию явно. Следующие операции эквивалентны:

```
titleA = 'Veni ' 'Vidi ' 'Vici '
titleB = 'Veni ' + 'Vidi ' + 'Vici '
```

В дополнение к универсальным операциям над последовательностями строки также имеют собственные операции, доступные в виде методов.

Например, строковый метод *find* представляет собой операцию поиска подстроки (возвращает смещение указанной подстроки или -1, если подстрока не найдена), а строковый метод *replace* выполняет глобальный поиск и замену. Существуют методы, которые разбивают строку на подстроки по разделителю, выполняют преобразования регистра символов, проверяют содержимое строки и удаляют пробельные символы из концов строки.

Метод *strip* убирает пробельные (или заданные) символы из начала и конца строки, метод *split* разбивает строку по указанному разделителю, методы *lower* и *upper* меняют регистр.

Строки в Python разрешено заключать в одинарные или двойные кавычки — они обозначают то же самое, но позволяют встраивать кавычки другого вида без отмены. То есть строковые литералы можно записывать заключенными либо в две одинарных, либо в две двойных кавычки — две формы работают одинаково и возвращают объект того же самого типа. Например, следующие две строки идентичны:

```
"Aut Caesar, aut nihil"
```

```
'Aut Caesar, aut nihil'
```

Кроме того, Python дает возможность заключать многострочные строковые литералы в утроенные кавычки (одинарные или двойные):

```
st = """Сегодня
```

```
взгляд наш
```

```
угрюм и кос"""
```

Такая строка называется *блочной строкой* и является синтаксическим удобством для написания многострочных текстовых данных. Строки, заключенные в утроенные кавычки, будут сохранять весь находящийся

между ними текст, в том числе любые символы справа от кода, которые могли задумываться как комментарии.

3.4. Булевские значения

Python имеет явный булевский тип данных по имени *bool* со значениями *True* и *False*. Имена *True* и *False* ведут себя в точности как целые числа 1 и 0 за исключением того, что они имеют специальную логику вывода — они отображают себя в виде слов *True* и *False*, а не цифр 1 и 0.

```
x = True
```

```
y = False
```

Тем не менее, реализация данного типа может привести к любопытным результатам. Так как *True* — целое число 1 со специальным форматом отображения, выражение «*True*+4» в Python дает целое число 5.

Кроме того, Python идентифицирует любую пустую структуру данных как ложь, а любую непустую — как истину.

То есть при проверке условия, следующие объекты будут считаться за истину: 'abc', [1, 2], {'a': 1}, 1. Ложными будут любые пустые объекты: "", [], {}, 0, 0.0, None.

3.5. неизменяемость.

Каждый объект в Python классифицируется как неизменяемый (немодифицируемый) или нет. Для основных типов справедливо, что числа, строки и кортежи неизменяемы, а списки, словари и множества — могут свободно модифицироваться на месте.

Например, нельзя изменить строку, присваивая значение символу в одной из ее позиций. Следующий пример вызовет ошибку:

```
A = 'Привет'
```

```
A[2] = 'е'
```

Чтобы изменить строку, обычно необходимо создать и присвоить новую строку с применением таких инструментов, как конкатенация и нарезание, и затем, при желании, присвоить результат первоначальному имени строки:

```
A = 'Привет'
```

```
A = A[:2] + 'е' + A[3:-1] + 'д'
```

```
print(A)          # 'Превед'
```

На самом деле здесь создается новая строка и присваивается переменной A, но выглядит как изменение исходной строки.

Аналогичных результатов можно добиться, используя вызовы методов вроде *replace*:

```
A = A.replace('и', 'е').replace('т', 'д')
```

Подобно любой операции, которая выдает новое строковое значение, строковые методы генерируют новые строковые объекты. Именно по этому в примере A было переприсвоено новое значение. Генерация нового строкового объекта для каждого изменения строки не настолько неэффективна, как может казаться. Python автоматически производит сборку мусора, поэтому новые объекты повторно занимают пространство, где хранились предшествующие значения.

Наоборот, изменяемые типы всегда могут модифицироваться на месте с помощью операций, которые не создают новые объекты. Несмотря на возможность копирования таких объектов, изменения на месте поддерживают прямую модификацию.

3.6. Переменные в Python

Переменные представляют собой просто имена, создаваемые, которые применяются для отслеживания информации в программе. Есть четыре основных пункта, которые нужно знать, для правильной работы с переменными:

- переменные создаются, когда им впервые присваиваются значения;
- переменные заменяются своими значениями, когда используются в выражениях;
- переменным должны быть присвоены значения до того, как их можно будет применять в выражениях;
- переменные ссылаются на объекты и никогда не объявляются заранее.

Если указать переменную, которой еще не присваивалось значение, то Python сообщит об ошибке, а не заполнит переменную каким-либо стандартным значением.

```
if x > 5:
```

```
...
```

```
NameError: name 'x' is not defined
```

Предварительно объявлять переменные в Python не требуется, но им хотя бы раз должны быть присвоены значения, прежде чем их можно будет применять. На практике это означает необходимость инициализировать счетчики нулем до того, как их можно будет увеличивать, инициализировать списки пустым списком, прежде чем их можно будет дополнять, и т.д.

Имена переменных должны начинаться с подчеркивания или буквы, за которой может следовать любое количество букв, цифр или подчеркиваний. Например, `_spam`, `spam` и `Spam__l` — допустимые имена, но `l_Spam`, `spam$` и `@#!` — нет.

Python — регистрозависимый язык. Это означает, что имена `x` и `X` относятся к двум различным переменным.

Имена переменных не могут совпадать со словами, которые означают особые вещи в языке Python. В таблице перечислены слова, которые в текущий момент являются зарезервированными.

Зарезервированные слова				
False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Переменные Python никогда не нужно объявлять заранее. Переменная создается, когда ей присваивается значение, которое может быть объектом любого типа, и она заменяется присвоенным значением, когда появляется в выражении. Переменной должно быть что-то присвоено ко времени использования ее значения.

4. Операторы и выражения

4.1. Выражения

Фундаментальным инструментом языка является *выражение* — комбинация чисел или других объектов и операций, которая вычисляет значение.

Выражения в Python записываются с использованием обычных математических обозначений и символов операций. Скажем, для сложения двух чисел `X` и `Y`, понадобится записать выражение `X+Y`, которое сообщит Python о необходимости применения операции `+` к значениям,

именованным как X и Y. Результатом выражения будет сумма X и Y, т.е. еще один числовой объект.

В таблице перечислены все операции выражений, доступные в Python.

Операции выражений	
Операции	Описание
yield x	Протокол send функции генератора
lambda арг.: вып.	Создание анонимной функции
x if y else z	Тернарный выбор
x or y	Логическое ИЛИ
x and y	Логическое И
not x	Логическое НЕ
x (not) in y	Членство
x is (not) y	Проверка идентичности объектов
x < y, <=, >, >=	Сравнение, проверка на надмножество и подмножество
x == y, x != y	Операции равенства значений
x y	Побитовое ИЛИ, объединение множеств
x ^ y	Побитовое исключающее ИЛИ, симметрическая разность множеств
x & y	Побитовое И, пересечение множеств
x << y, x >> y	Побитовый сдвиг
x + y	Сложение, конкатенация
x - y	Вычитание, разность множеств
x * y	Умножение, повторение
x % y	Остаток, формат
x / y, x // y	Деление, деление с округлением
-x, +x	Противоположность, идентичность

Операции выражений	
Операции	Описание
$\sim x$	Побитовое НЕ (инверсия)
$x ** y$	Возведение в степень
$x[i]$	Индексация
$x[i:j:k]$	Нарезание
$x(...)$	Вызов(функции, класса...)
$x.атрибут$	Ссылка на атрибут
$(...)$	Кортеж, выражение, выражение генератора
$[...]$	Список, списковое включение
$\{...\}$	Словарь, множество, включения множеств и словарей

Как и в других языках, сложные выражения в Python записываются путем объединения операций из таблицы, например:

$a * b + c * d$

4.2. Порядок выполнения операций

Смешанные операции следуют *старшинству операций*.

Операции в таблице упорядочены по старшинству:

- операции, находящиеся ниже имеют более высокий приоритет;
- операции, расположенные в одной строке, обычно группируются слева направо (кроме возведения в степень).

Например, с обычным приемом проверки человека на внимательность $2+2*2$ Python с легкостью справляется: первым выполнит умножение, а затем сложение результата и двойки, потому что операция $*$ имеет более высокий приоритет.

Заключение подвыражений в круглые скобки приводит к переопределению правил старшинства Python; выражения в круглых скобках всегда вычисляются первыми и затем их результаты используются в охватывающих выражениях.

4.3. Преобразование типов в выражениях

Помимо смешивания операций в выражениях допускается также смешивать числовые типы. Скажем, можно сложить целое число и число с плавающей точкой. В выражениях с разнородными числовыми типами Python сначала преобразует операнды к типу самого сложного операнда и затем выполняет операции над операндами того же самого типа. Такое поведение похоже на преобразования типов в языке C.

```
x = 5
```

```
y = 3.5
```

```
z = x + y # 8.5
```

Вызывая встроенные функции, типы можно преобразовывать вручную. Например, следующая строка выведет 3:

```
print(int(3.1415))
```

Как правило, в этом нет нужды: поскольку Python автоматически преобразовывает операнды в более сложный тип внутри выражения.

Все преобразования между разнородными типами применяются только при смешивании в выражении числовых типов. В общем случае Python не выполняет автоматическое преобразование между другими типами. Скажем, добавление строки к целому числу приводит к ошибке, если только не преобразовать одно в другое вручную.

4.4. Операции сравнения

Операции сравнения работают для чисел вполне ожидаемым образом — они сравнивают относительные величины своих операндов и возвращают булевский результат, который в дальнейшем можно проверять для принятия решения в более крупном операторе.

Python также позволяет выстраивать в цепочку множество сравнений для выполнения проверок вхождения в диапазон. Такие сравнения называются *сцепленными* и являются краткой записью для более крупных булевских выражений. Например:

```
x < y < z
```

```
x < y and y < z
```

Обе строки кода выполняют одну и ту же проверку.

В сцепленных проверках можно использовать другие сравнения, но результирующие выражения могут стать менее понятными. К тому же разрешено выстраивать цепочки произвольной длины:

```
x == y < z != w
```

Данная строка означает `x == y and y < z and z != w`.

Еще один важный момент: числа с плавающей точкой могут не всегда работать ожидаемым образом и для содержательного сравнения требовать преобразований или других манипуляций:

```
1.1 + 2.2 == 3.3 # False
```

```
1.1 + 2.2 # 3.3000000000000003
```

Причина связана с тем фактом, что числа с плавающей точкой не способны представлять определенные значения точно из-за ограниченного количества битов — фундаментальная проблема в численном программировании, не уникальная для языка Python.

Одним из возможных решений такой проблемы, является использование функций округления *round*, *floor*, *ceil*, *trunc* или функций преобразования типа перед сравнением:

```
round(1.1 + 2.2, 1) == 3.3 # True
```

4.5. Операции деления

В Python 3.X одна косая черта означает операцию *настоящего* деления. Оно всегда сохраняет дробные части в результатах с плавающей точкой независимо от типов операндов. (Важно, что в версиях языка Python 2.X такая операция выполняет *классическое* деление, усекая результат для целых чисел и сохраняя дробные части для чисел с плавающей точкой подобно языку C.)

Также существует операция *деления с округлением* в меньшую сторону (другое название — операция *усекающего деления*). Данная операция всегда усекает дробные остатки, округляя в меньшую сторону независимо от типов операндов. Тип результата зависит от типов операндов.

Пример работы на Python 3.X:

```
10/4          # 2.5 – настоящее деление
```

```
10/4.0 # 2.5 – настоящее деление
```

```
10//4  # 2 – с округлением вниз
```

```
10//4.0 # 2.0 – с округлением вниз
```

Две последние строки кода показывают, как изменяется тип результата вместе с изменением типа операндов.

Остаток от деления вычисляется операцией %.

5. Задачи

5.1. Пример решения

Задача 1. При заданных в коде значениях a , b и c , решить уравнение $\frac{x-a}{x-b} = c$. Вывести на консоль точный результат и результат, округленный до целого значения.

Решение:

```
# Инициализация коэффициентов
```

```
a = 3
```

```
b = 4.12
```

```
c = -2.5
```

```
# Решение
```

```
x = (a - b * c) / (1 - c)
```

```
# Округление
```

```
r = round(x)
```

```
# Печать
```

```
print(x)
```

```
print(r)
```

```
# Пустой input
```

```
input()
```

Описание.

Создается текстовый файл с расширением `.py`, в который помещается вышеприведенный код.

В первой части задачи объявляются переменные a , b и c и присваиваются их значения. Какие бы значения здесь ни были присвоены,

результат решения должен быть верным. В данном случае переменная `a` была объявлена как целочисленная, а `b` и `c` — как числа с плавающей точкой. Но они могут иметь любой числовой тип.

После этого, по правилам математики был выражен `x` и записан в формулах языка Python.

Функция `round` осуществляет округление по правилам математики. Ее результат помещается в отдельную переменную `r`, не меняя значение `x`.

Функции `print(x)` и `print(r)` выводят на консоль точное значение `x` и округленное значение `r` одно под другим.

Вызов функции `input` в конце задачи — стандартный прием работы с консолью для многих языков программирования. Он нужен для того, чтобы консоль не закрывалась сразу, а ожидала ввода (или закрытия), давая возможность посмотреть выведенную информацию.

Теперь, при нажатии двойным щелчком по созданному файлу, при условии, что Python установлен, а код написан без ошибок, будет открываться консоль, выводящая результат работы данной программы.

Задача 2. В коде дана строка `s`. Создать строку, в которой первая и последняя трети берутся из исходной строки, а вторая треть заменятся на соответствующее число символов `*` (звездочка). Вывести исходную строку и строку-результат.

Решение:

```
# Исходная строка
```

```
s = 'Абросим не просит, а дадут – не бросит'
```

```
# Решение
```

```
size = len(s) # Длина строки
```

```
# Первая треть
```

```
pt1 = s[:round(size / 3)]
```

```

# Последняя треть
pt3 = s[round(2 * size / 3):]

# Строка из звездочек
mid = '*' * (size - len(pt1) - len(pt3))

# Результирующая строка
res = pt1 + mid + pt3

# Вывод
print(s)
print(res)

# Пустой input
input()

```

Описание.

В начале задачи объявлена строка `s`. Далее в переменную `size` помещается длина исходной строки.

Первая и последняя трети строки берутся с помощью операции нарезания. Операция округления выполняется, так как длина строки может быть не кратна трем, а для операции индекса или нарезания требуются целые числа.

Далее с помощью операции повторения создается строка `mid` из звёздочек. Она имеет длину, которая вычисляется как разность исходной длины и двух уже полученных частей строки.

После этого с помощью операций конкатенации (склеивания) формируется строка-результат. Для этого берется первая треть строки `pt1`, строка из звездочек `mid` и последняя треть строки `pt3`.

В конце задачи выводятся исходная строка, строка-результат, и стоит пустой ожидающий `input`.

5.2. Арифметические задачи

1. При заданных в коде вещественных x , y и z найти значение выражения $\frac{x^y + y^z}{z^x}$. Вывести на консоль точный результат и результат, округленный до целого значения.
2. При заданных в коде вещественных a , b и c найти дискриминант $(x + a)^2 + (x + b)^2 = cx^2$. Вывести на консоль точный результат и результат, округленный до целого значения.
3. При заданных в коде вещественных $x \geq 0$, y и $z \geq 0$ найти значение выражения $\sqrt[3]{x} - y\sqrt[3]{z}$. Вывести на консоль точный результат и результат, округленный до целого значения.
4. При заданном в коде целом числе n , найти сумму следующих двух значений: количество сотен в числе и количество единиц. Например, в числе 1024 одна тысяча, ноль сотен, два десятка и четыре единицы.
5. При заданных в коде целых числах x , y и z найти остаток деления $2^x + 3^y$ на z .
6. При заданных в коде вещественных x , y и z найти результаты настоящего и целочисленного делений $x^2 + y$ на 2^z .
7. При заданных в коде вещественных x , y и z вывести модуль их суммы и сумму их модулей.
8. При заданных в коде вещественных a и b найти решение уравнения $x^a = b$, при условии, что $x \geq 0$.
9. При заданных в коде вещественных x , y и z найти значение выражения $-|x| + 2y + z^4$. Вывести на консоль точный результат и результат, округленный до целого значения.
10. При заданных в коде вещественных a , b , c и x найти значение выражения $\frac{|a|x^{-5}}{|b|x^c}$. Вывести на консоль точный результат и результат, округленный до целого значения.

5.3. Задачи на строки

1. В коде дана строка *s*. Создать строку, в которой все символы, кроме первого, последнего и символов второй трети строки, заменяются на символ + (плюс). Вывести исходную строку и строку-результат.
2. В коде дана строка *s*. Создать строку, в которой первая, третья и пятая части берутся из исходной строки, а вторая и четвертая — заменяются на соответствующее число символов # (шарп). Вывести исходную строку и строку-результат.
3. В коде дана строка *s*. Создать строку, в которой все четверти исходной строки идут в обратном порядке. То есть сначала последняя четверть строки, затем третья, потом вторая и первая. Вывести исходную строку и строку-результат.
4. В коде даны строки *a* и *b*. Создать строку «строка_а + строка_b = строка_астрока_b» включая пробельные символы, символы + и =. Вывести исходные строки и строку-результат.
5. В коде дана строка *s*. Из нее создать две: строку без первых 3 и последних 2 символов, строку из 3 первых символов, повторяющихся 2 раза, и 2 последних символов, повторяющихся 3 раза. Вывести исходную строку, строки 1 и 2 и длину строки 1.
6. В коде дана строка *s*, не превышающая 80 символов в длину. Дополнить ее символами _ (нижнее подчеркивание) до 80 символов по следующему правилу: в результат 1 поместить исходную строку по центру, в результат 2 — справа. Вывести исходную строку, и оба результата.
7. В коде даны два числа *x* и *y*. Вывести строку «значение_x + значение_y = их_сумма» с помощью методов приведения типа и конкатенации. Например «3 + 4 = 7». Вывести длину полученной строки.

8. В коде дана строка *s* в нижнем регистре. Вывести исходную строку, строку, где первый символ в верхнем регистре, затем всю строку в верхнем регистре.
9. В коде даны строки *s* и *x*. Вывести сначала число вхождений строки *x* в строку *s*, затем из начала и конца строки *s* удалить символы строки *x*, и вывести результат.
10. В коде дана строка *s* в верхнем регистре. Вывести исходную строку, строку, где только первый символ находится в верхнем регистре, затем всю строку в нижнем регистре.

6. Консольный ввод-вывод

6.1. Управляющие последовательности

Обратная косая черта в строках используется для введения специальных кодировок символов, известных как *управляющие последовательности*.

Управляющие последовательности позволяют внедрять в строки символы, которые не могут быть легко набраны на клавиатуре. Символ «\» и один или более следующих за ним символов в строковом литерале в результирующем строковом объекте заменяются одиночным символом, который имеет двоичное значение, указанное управляющей последовательностью. Например:

```
text = 'A\tB' # результат: A B
```

Длина данной строки, получаемая операцией *len* равна 3, так как «\t» заменяется одним символом табуляции.

Для кодирования таких специальных символов Python распознает полный набор управляющих последовательностей; основные из них приведены в таблице.

Символы, представляемые в строке посредством обратной косой черты	
Управляющая последовательность	Описание
<code>\\</code>	Обратный слеш
<code>\'</code>	Одинарная кавычка
<code>\"</code>	Двойная кавычка
<code>\n</code>	Новая строка
<code>\t</code>	Горизонтальная табуляция
<code>\v</code>	Вертикальная табуляция
<code>\xhh</code>	Символ с шестнадцатеричным значением
<code>\ooo</code>	Символ с восьмеричным значением

В Python нулевой (пустой) символ «\0» не завершает строку, как нулевой байт обычно делает в языке C. Взамен Python хранит в памяти и длину, и текст строки. На самом деле в Python не предусмотрен какой-либо символ для завершения строк.

Если Python не распознает символ после обратной косой черты в качестве допустимого управляющего кода, то он просто оставляет обратный слеш в результирующей строке:

```
text = 'C:\py\code'
```

Однако в подобных примерах не следует полагаться на такое поведение. Правильно будет явно удваивать обратный слеш, иначе можно допустить ошибку:

```
text = 'C:\\py\\text'
```

Результатом будет присваивание переменной `text` строки «C:\py ext».

Иногда специальная трактовка обратной косой черты для введения управляющей последовательности может вызывать затруднения. Проблема

такого рода, при передаче имен файлов, была показана в последнем примере.

В таких ситуациях становятся полезными *неформатированные строки*. Если прямо перед открывающей кавычкой строки указать букву «r» (в нижнем или верхнем регистре), то механизм отмены отключается. В итоге Python сохраняет символы обратной косой черты буквально, в точности как они набраны. Например:

```
text = r'C:\py\text'
```

Результатом этой строки кода будет присваивание переменной `text` уже правильной строки «C:\py\text». Длина этой строки естественным образом будет равна 10.

Несмотря на свою роль, даже в неформатированной строке нужно экранировать символ кавычки. С этим связана еще одна трудность — строка не может заканчиваться одиночным обратным слешем (или нечетным их числом), поскольку он экранирует следующий за ним символ кавычки. То есть `r'...\'` не является допустимым строковым литералом.

6.2. Преобразования

Если сценарий получает число в виде текстовой строки, то перед тем, как строку можно будет трактовать как число или наоборот, необходимо задействовать инструменты преобразования. Например:

```
A = '42' + 1      # Ошибка
```

```
B = int('42') + 1 # 43
```

```
C = '42' + str(1)  # '421'
```

Для преобразования типа к числу с плавающей точкой используется встроенная функция *float*.

Также возможно преобразовывать одиночный символ в его внутренний целочисленный код, передавая символ встроенной функции *ord*, которая

возвратит действительное двоичное значение, используемое для представления соответствующего символа в памяти. Функция *chr* выполняет обратное преобразование, принимая целочисленный код и возвращая надлежащий символ:

```
ord('s'))      # 115
```

```
chr(115)       # 's'
```

6.3. Форматирование строк

Форматирование строк позволяет выполнять множество подстановок, специфических для типов, в строке за единственный шаг. Оно не является строго обязательным, но может быть удобным, особенно когда формируемый текст подлежит отображению для пользователей программы.

В Python форматирование строк доступно в двух вариантах.

Первый способ, доступный с момента появления Python, основан на модели *printf* языка C и называется *выражением форматирования строк*.

В Python определена бинарная операция % для работы со строками. Она предлагает компактный способ записи для множества подстановок строк одновременно вместо создания и конкатенации отдельных частей.

Чтобы сформатировать строку, необходимо выполнить следующие действия:

1. Слева от операции % записывается строка формата, содержащая одну или более встроенных целей преобразования, каждая из которых начинается с %.

2. Справа от операции % указывается объект или объекты, внедренные в кортеж, которые необходимо вставить в строку формата слева на месте целей преобразования.

Например:

```
A = 'черненьких'
```

```
B = '%d %s %s чертенка' % (4, A, 'чумазеньких')
```

В результате выполнения данного кода, переменная B будет указывать на строку «4 черненьких чумазеньких чертенка». Число 4 подставляется на место %d, затем на место первого %s подставляется строка, хранящаяся в A, и наконец, вместо %s подставляется строка «чумазеньких».

В следующей таблице представлены основные коды типов для форматирования строк. Они записываются после символа %.

Коды типов в форматировании строк	
Код	Описание
s	Строка
c	Символ (int или str)
d	Целое с основанием 10
i, u	Целое число
o	Восьмеричное число
x, X	Шестнадцатеричное число
e, E	Число с плавающей точкой со степенью
f, F	Десятичное число с плавающей точкой
g, G	Число с плавающей точкой e или f
%	Символ % (записывается как %%)

Программисты на C узнают большинство из этих кодов, потому что форматирование строк Python поддерживает все обычные коды формата *printf* языка C.

На самом деле цели преобразований в строке формата в левой части выражения поддерживают различные операции преобразования с довольно

сложным синтаксисом. Общая структура целей преобразований выглядит следующим образом:

`%[(имя_ключа)][флаги][ширина][.точность]код_типа`

Символы типов кодов находятся в конце такого формата целевой строки. Между % и символом кода типа можно делать любое из описанных ниже действий:

- указывать имя ключа для индексации словаря, используемого в правой части выражения;
- перечислять флаги («-» — выравнивание влево, «+» — знак числа, пробел — пусто перед положительными числами и минус для отрицательных чисел, «0» — дополнение нулями);
- задавать общую минимальную ширину поля для подставляемого текста
- устанавливать количество цифр, отображаемых после десятичной точки (точность), для чисел с плавающей точкой.

Например:

`P = 3.1415926535`

`A = 'pi = %d' % P # '3'`

`B = 'pi = %f' % P # '3.141593'`

`C = 'pi = %15f' % P # ' 3.141593'`

`D = 'pi = %015f' % P # '00000003.141593'`

`E = 'pi = %.3f' % P # '3.142'`

В качестве более развитого расширения форматирование строк также разрешает целям преобразования в левой части ссылаться на ключи в словаре, записанном в правой части, и извлекать соответствующие значения:

```
A = '%(tiger)s, %(tiger)s, жгучий %(fear)s' %
```

```
    {'fear': 'страх', 'tiger': 'тигр'}
```

Результат — строка «тигр, тигр, жгучий страх». Следует обратить внимание, что при такой записи порядок и количество целей слева и объектов справа могут не совпадать. Словари заключаются в фигурные, а не круглые скобки.

Второй способ форматирования, появившийся в версиях Python 2.6 и 3.0, называется *вызов методов форматирования строк*.

Метод *format* строкового объекта базируется на обычном синтаксисе вызова функций, а не на выражении. В частности, он использует в качестве шаблона строку, на которой вызывается, и принимает любое количество аргументов, которые представляют значения, подлежащие подстановке согласно шаблону.

Фигурные скобки внутри строки, на которой метод вызывается, обозначают цели подстановки и аргументы, вставляемые по позиции, по ключевому слову или по относительной позиции. Похожий синтаксис можно встретить, например, в языке C#.

В результате следующего кода, В будет ссылаться на строку «Что дозволено Юпитеру, не дозволено быку»:

```
A = 'Что дозволено {0}у, не дозволено {1}у'
```

```
B = A.format('Юпитер', 'бык')
```

За счет добавления дополнительного синтаксиса в строку формата можно достичь более специфических результатов. Общая структура такова:

```
{имя_поля компонент ! флаг_преобразования :  
 спецификатор_формата}
```

- имя поля — необязательное число или ключевое слово, идентифицирующее аргумент, которое может быть опущено для применения относительной нумерации аргументов;

- компонент — строка из нуля или большего числа ссылок *.имя* либо *[индекс]*, используемых для извлечения атрибутов и индексированных значений аргумента, которая может быть опущена для применения всего значения аргумента;

- флаг преобразования, если присутствует, начинается с символа «!», за которым следует r, s или a для вызова назначения встроенных функций *repr*, *str* или *ascii* соответственно;

- спецификатор формата, если присутствует, начинается с символа «:», за которым следует текст, указывающий, каким образом значение должно быть представлено, в том числе такие детали, как ширина поля, выравнивание, дополнение, десятичная точность и так далее, и в конце необязательный код типа данных.

В следующем примере в строке формата «0» — позиция аргумента, «<» - выравнивание по левому краю (для выравнивания по правому краю используется символ «>»), «12» — общая ширина поля (7 цифр, 1 точка и 4 пробела) и «.7» — точность (число знаков после запятой минус один):

```
p = 3.1415926535
```

```
a = 'Пи = {0:<12.7}'.format(p)
```

```
# 'Пи = 3.141593      '
```

Числа с плавающей точкой поддерживают в вызовах методов форматирования те же самые коды типов и особенности форматирования, что и в выражениях %.

Метод *format* также поддерживает шестнадцатеричный, восьмеричный и двоичный форматы.

6.4. Функции `print` и `input`

С точки зрения синтаксиса вызов функции `print` в Python3.X имеет следующую форму:

```
print([object, ...] [, sep=' '] [, end='\n']  
[, file=sys.stdout] [, flush=False])
```

В показанной формальной записи элементы в квадратных скобках являются необязательными и в заданном вызове могут быть опущены, а значения после символа `=` предоставляют стандартные значения для аргументов.

То есть встроенная функция `print` выводит в поток данных `file` текстовые представления одного или большего числа объектов `object`, разделенные строкой `sep`, за которыми следует строка `end`, сбрасывая или не сбрасывая буферизированный вывод согласно аргументу `flush`.

Текстовое представление каждого выводимого объекта `object` получается путем передачи объекта встроенной функции `str`.

Например:

```
i = 100; s = 'text'; f = 2.7182  
  
print(i, s, f, sep=' --- ', end='')
```

В результате три объекта `i`, `s` и `f` будут напечатаны в строку через разделитель « --- ». Присвоение пустой строки параметру `end` отключает стандартный перевод каретки на новую строку после печати.

Параметр `file` указывает файл, стандартный поток данных или другой объект, подобный файлу, в который будет отправляться текст; если он не передан, то по умолчанию принимается поток стандартного вывода `sys.stdout`. Файлы должны быть уже открыты для вывода, перед вызовом метода `print`.

Функция *input* считывает и возвращает строку входных данных и имеет следующий синтаксис:

```
input([prompt])
```

Где необязательный параметр *prompt* — строка-приглашение, которая будет отправлена на стандартное устройство вывода:

```
x = input('Введите строку: ')
```

Функция возвращает строковый тип, поэтому, для получения данных других типов, требуется явно ее преобразовывать.

7. Ветвления и циклы

7.1. Ветвления

Оператор *if* является главным инструментом выбора в Python. В своей полной форме оператор может также иметь конструкцию *else* для стандартного случая и одну или более конструкций *elif* (что означает «else if») для других проверок.

Оператор *if* языка Python компактен, по сравнению с другими СИ-подобными языками, но имеет один дополнительный компонент — двоеточие.

При этом, круглые скобки, в которые можно заключить условие, необязательны, но и ошибкой не являются. То, что конец строки является концом оператора, а отступ — заменяет блок, заключенный в фигурные скобки, добавляет компактности коду.

Кроме того, оператор может иметь более двух вариантов выбора: сначала записывается часть *if* с условным выражением, далее могут следовать одна или более необязательных частей *elif*, и, наконец, необязательная часть *else*. Общая форма записи условной инструкции *if* выглядит следующим образом:

```
if усл_1:
    оп_1
elif усл_2:
    оп_2
else:
    оп_3
```

Конструкция с несколькими `elif` может также служить отличной заменой конструкции `switch - case` в других языках программирования.

Следует еще раз отметить, что, если в блоке операторов должно выполняться более одного действия, тогда нужно просто обеспечить всем им одинаковые отступы.

В случае нескольких вложенных операторов `if`, блок `else` относится не к последнему оператору `if` (как в языке C), а к соответствующему по отступам.

7.2. Логические операторы

Булевские операции возвращают объект истинного или ложного операнда и прекращают оценку, как только результат становится известным.

В Python существуют три булевских операции выражений:

`x and y`

`x or y`

`not x`

Операция `and` — логическое И — истина, если оба значения `x` и `y` истинны.

Операция `or` — логическое ИЛИ — истина, если хотя бы одно из значений `x` или `y` истинно.

Операция `not` — логическое НЕ — истина, если `x` ложно.

Булевские операции в Python набираются как слова (в отличие от `&&`, `||` и `!` в языке C). Кроме того, важно понимать, что операции `and` и `or` в Python возвращают истинный или ложный объект, а не значение `True` или `False`:

```
2 < 3 # Результат True
```

```
2 or 3 # Результат 2
```

Поскольку любой ненулевой объект является истиной, а операции `and` и `or` возвращают именно объекты, результатом последней строки является число 2.

7.3. Трехместное выражение `if else`

Существует более компактная версия оператора ветвления — трехместное выражение `if else`.

```
a = b if x else c
```

Если `x` истинно, выполнится команда `a = b`, в противном случае — выполнится выражение `a = c`.

7.4. Циклы

Цикл *for* является простым и эффективным способом прохода по всем элементам в любом итерируемом объекте и выполнения блока кода для каждого элемента по очереди.

Объект считается *итерируемым*, если он является либо физически сохраненной последовательностью в памяти, либо объектом, который генерирует по одному элементу за раз в контексте итерационной операции — своего рода «виртуальной последовательностью».

Цикл *for* и более универсальный цикл *while* предоставляют главные способы написания кода для выполнения повторяющихся задач в виде операторов в сценариях. Тем не менее, в действительности цикл *for*

является операцией над последовательностью. Он работает на любом объекте, относящемся к последовательностям и даже на некоторых вещах, не считающихся последовательностями. Цикл *for* чуть менее универсальный, но выполняется гораздо быстрее цикла *while*.

Пример использования цикла:

```
for i in 'hello there':  
    print(i * 2, end='')
```

В результате выполнения примера будет выведено сообщение: «hheelllloo tthheerree».

Для повторения цикла некоторое заданное число раз *n* можно использовать цикл *for* вместе с функцией *range*:

```
for i in range(3):  
    print(i)
```

Функция *range* может также принимать не один, а два или три параметра. Вызов *range(a, b)* означает, что индексная переменная будет принимать значения от *a* до *b* - 1. Третий параметр означает величину изменения (шаг) индексной переменной.

Цикл *while* языка Python представляет собой более универсальную разновидность инструментов для организации циклов; он не ограничивается проходом через последовательности, но в целом требует написания большего объема кода.

Тело цикла выполняется до тех пор, пока условие цикла истинно.

```
i = 0  
  
while i < 10:  
    print(i)  
    i = i + 1
```

Оператор *break* переходит за пределы ближайшего заключающего его цикла, то есть начинают выполняться строки кода после всего оператора цикла, в котором расположен этот *break*.

Оператор *continue* переходит в начало ближайшего заключающего цикла на строку заголовка цикла, то есть позволяет игнорировать все тело цикла, находящееся после *continue*.

Слово *else*, примененное в цикле *for* или *while*, проверяет, был ли произведен выход из цикла инструкцией *break*, или же естественным образом. Блок инструкций внутри *else* выполнится только в том случае, если выход из цикла произошел без помощи *break*.

```
for i in 'hello world':  
    if i == 'a':  
        break  
    else:  
        print('Буквы а в строке нет')
```

8. Последовательности и отображения

8.1. Списки

Объект списка Python является наиболее общей последовательностью, предлагаемой языком. Списки представляют собой позиционно упорядоченные коллекции объектов произвольных типов и не имеют фиксированных размеров. Кроме того, они изменяемы — в отличие от строк списки можно модифицировать на месте путем присваивания по смещениям и вызова разнообразных списковых методов. Соответственно они предоставляют очень гибкий инструмент для представления произвольных коллекций.

Создание списков происходит следующим образом:

```
A = ['a', 'b', 'c', 'd']
```

```
B = list('abcd') # Создание из строки
```

Оба списка будут одинаковыми.

Будучи последовательностями, списки поддерживают все операции над последовательностями, которые описаны для строк (такие как конкатенация и повторение).

Списки Python могут напоминать массивы в других языках, но они, как правило, более мощные. Прежде всего, отсутствует ограничение, что элементы должны принадлежать какому-то фиксированному типу. Кроме того, списки не имеют фиксированных размеров. То есть они могут увеличиваться и уменьшаться в ответ на операции, специфичные для списков.

Списки поддерживают множество методов:

- метод *append* увеличивает размер списка и помещает объект в конец (в отличие от конкатенации с одним элементом, метод *append* не генерирует новые объекты, поэтому работает быстрее);

- метод *pop* удаляет и возвращает последний элемент (или элемент по указанной позиции);

- оператор *del* удаляет элемент по заданному смещению, приводя к уменьшению списка;

- метод *insert* вставляет объект в произвольную позицию;

- метод *remove* удаляет указанный элемент по значению;

- метод *extend* добавляет множество элементов в конец;

- метод *sort* осуществляет сортировку списка;

- метод *reverse* обращает порядок элементов;

- метод *copy* создает копию списка;

- метод *clear* очищает список;

- метод *index* возвращает позицию объекта;

- метод *count* возвращает число вхождений.

Поскольку списки являются изменяемыми, большинство списковых методов также модифицируют объект списка на месте, а не создают новый такой объект. Например, метод *sort* по умолчанию упорядочивает список по возрастанию, а метод *reverse* обращает его — в обоих случаях методы модифицируют список напрямую:

```
A = ['a']
```

```
B = A.append('b') # A = ['a', 'b'] B = None
```

```
A = A.append('c') # A = None B = None
```

В результате выполнения последней строчки, ссылка на список вообще утратится.

Хотя списки не имеют фиксированных размеров, Python все же не разрешает ссылаться на несуществующие элементы. Это называется *контроль границ*. Индексация и присваивание за концом списка всегда считается ошибкой. При этом списки способны увеличиваться и уменьшаться на месте.

Извлекать объект компонента из списка можно путем индексации списка по смещению объекта:

```
arr = ['a', 'b', 'c', 'd', 'e']
```

```
x = arr[3] # x = 'd' (нумерация с нуля)
```

Дополнительное удобство основных типов данных Python заключается в том, что они поддерживают произвольное *вложение* — можно вкладывать типы в любой комбинации и на любую желаемую глубину. Одно из прямых применений такой возможности связано с представлением в Python *матриц*, то есть многомерных массивов:

```
matr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

В примере создается список, который содержит три других списка. Результатом будет представление матрицы 3x3 чисел. Доступ к такой структуре может осуществляться разнообразными способами:

```
matr[0]      # Получение строки
```

```
matr[1][2]   # Получение элемента
```

В дополнение к операциям над последовательностями и списковым методам в Python имеется более сложная операция, известная как *выражение спискового включения* (list comprehension). Следующий пример показывает, как с помощью этой операции можно извлечь столбец матрицы:

```
col = [row[2] for row in matr]
```

Списковые включения являются способом построения нового списка за счет выполнения выражения на каждом элементе в последовательности, по одному за раз, слева направо. Списковые включения могут выполнять более сложные операции:

```
col = [row[2]*4 for row in matr]
```

В данном примере не только выберется третий столбец, но и каждый его элемент будет умножен на 4.

Формально, списки Python содержат ноль и более ссылок на другие объекты. Они могут напоминать массивы указателей (адресов) в других языках. Поэтому извлечение элемента из списка Python производится почти также быстро, как индексация массива языка C.

Операции индекса и среза используются не только для извлечения, но и для присваивания элементов:

```
A = ['a', 'b', 'c']
```

```
A[0:2] = ['d', 'e']    # A = ['d', 'e', 'c']
```

Длина последовательности, которая присваивается, не обязана совпадать с длиной присваиваемого среза. В следующем примере срез [0:1], равный одному элементу «1», заменяется двумя элементами:

```
B = [1, 2]
B[0:1] = [3, 4]      # B = [3, 4, 2]
```

8.2. Словари

Словари Python — нечто совершенно иное; они вообще не являются последовательностями и взамен известны как *отображения*. Отображения также представляют собой коллекции других объектов, но они хранят объекты по ключам, а не по относительным позициям. В действительности отображения не поддерживают какой-либо надежный порядок слева направо; они просто отображают ключи на связанные значения. Словари — единственный тип отображения в наборе основных объектов Python — являются изменяемыми: как и списки, их можно модифицировать на месте, и они способны увеличиваться и уменьшаться по требованию.

При написании в виде литералов, словари указываются в фигурных скобках и состоят из ряда пар *ключ: значение*. Словари удобны всегда, когда необходимо ассоциировать набор значений с ключами — например, для описания свойств чего-нибудь.

Вот простейший пример создания и использования словаря:

```
d = {'one': 1, 'two': 2, 'three': 3}
print(d['two'])
```

В результате выполнения такой программы будет выведено число 2. Операция индексации словаря имеет такой же синтаксис, как для последовательностей, но элементом в квадратных скобках будет ключ, а не относительная позиция.

В отличие от присваивания элементу в списке, находящемуся вне установленных границ, которое запрещено, присваивание новому ключу словаря приводит к созданию этого ключа:

```
d['four'] = 4
```

Словари могут использоваться также для замены операций поиска — индексирование словаря по ключу зачастую оказывается самым быстрым способом реализации поиска в Python.

Альтернативным способом создания словаря является использование функции *dict* с аргументами в виде *ключ = значение* или в виде списка из кортежей:

```
d = dict(one=1, two=2, three=3)
```

```
d2 = dict([('one', 1), ('two', 2), ('three', 3)])
```

Следует также отметить отсутствие кавычек в первом случае у строк-ключей словаря.

Еще один важный момент — это перепутывание порядка элементов, порядок слева направо для ключей словаря не гарантирован. Отображения не являются позиционно упорядоченными, поэтому они возвратятся в порядке, отличающемся от того, в котором были набраны.

Подобно спискам словари могут быть вложены.

Несмотря на возможность выполнять присваивание новому ключу для расширения словаря, извлечение несуществующего ключа по-прежнему расценивается как ошибка. Выражение проверки членства в словаре, *in*, позволяет запрашивать существование ключа и организовать ветвление в зависимости от результата с помощью оператора *if*. Следующий код выведет элемент по ключу *five*, если ключ присутствует в словаре *d*:

```
if 'five' in d:
    print(d['five'])
```

Словари поддерживают множество методов:

- метод *keys* возвращает все ключи словаря;
- метод *values* возвращает все значения;
- метод *items* возвращает все кортежи вида (ключ, значение);
- метод *copy* создает копию словаря;
- метод *clear* очищает словарь;
- метод *update* объединяет по ключам с указанным словарем (если ключи совпадут, значения будут переписаны);
- метод *get* извлекает по ключу;
- метод *pop* удаляет по ключу;
- метод *setdefault* устанавливает значение по ключу;
- метод *popitem* удаляет и возвращает пару ключ-значение.

8.3. Кортежи

Объект кортежа похож на список, который нельзя изменять — кортежи являются последовательностями подобно спискам, но они неизменяемые подобно строкам.

Синтаксически они записываются в круглых, а не квадратных скобках и поддерживают произвольные типы, произвольное вложение и обычные операции над последовательностями.

```
t1 = (1, 2, 3)
```

```
t2 = (1, )
```

В примере показано создание кортежей *t1* и *t2*. Одноэлементные кортежи требуют написание хвостовой запятой.

На практике кортежи применяются в целом не так часто, как списки, но они обеспечивают своего рода ограничение целостности: если передавать коллекцию объектов внутри программы в виде списка, тогда

он может быть модифицирован где угодно; если использовать кортеж, то изменить его не удастся.

Кортежи не поддерживают столько методов как списки, но они разделяют с ними большинство свойств: упорядоченность, доступ по смещению (индексу), допускают вложения, разнородны (могут хранить одновременно элементы различных типов).

Альтернативное создание кортежа, например из строки, осуществляется методом `tuple`:

```
t3 = tuple('abc')
```

9. Задачи

9.1. Пример решения

Задача 3. В коде даны 6 чисел с плавающей точкой. Создать строку формата, в которой эти числа будут располагаться в 2 столбца: в первом у всех чисел 2 знака после запятой и выравнивание по левому краю, во втором — 4 знака после запятой, выравнивание вправо и указана единица измерения «Ампер». Вывести данную строку.

Решение:

```
# Произвольные заданные значения
```

```
n0 = 5.4784451
```

```
n1 = 45.488
```

```
n2 = -4275.145578
```

```
n3 = 7.445
```

```
n4 = 85
```

```
n5 = 369 / 452 # 0.8163716814159292
```

```
# Формирование строки формата (одна строка кода)
```

```

result = '%-12.2f %12.4f Ампер\n%-12.2f %12.4f
Ампер\n%-12.2f %12.4f Ампер' % (n0, n1, n2, n3, n4, n5)

# Вывод

print(result)

input()

```

Описание.

В начале задачи просто заданы 6 чисел.

Далее создается строка `result` с помощью оператора выражения форматирования строк `%`.

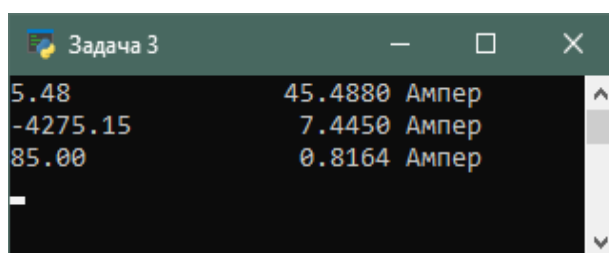
Первая часть «`%-12.2f`» помещает в строку число с плавающей точкой (`%f`), которое имеет два знака после запятой (`.2`). Всего под число выделено 12 символов в строке, где 12 — просто достаточное для вывода число символов. Выравнивание влево указано флагом «`-`».

Следующая часть «`%12.4f Ампер`» помещает в ту же строчку второе число через пробел, которое имеет четыре знака после запятой (`.4`) и выравнивание вправо (значение выравнивания по умолчанию). Затем указана единица измерения.

После этого происходит перевод каретки (`\n`), после чего все рассмотренные части повторяются дважды для строк 2 и 3.

После оператора `%` в круглых скобках перечислены все подставляемые значения. В конце задачи строка выводится на консоль.

Результат.



Задача 4. Пользователь вводит целое число n . Создать список из 100 элементов. Первые n значений — натуральные числа от 1 до n . Остальные элементы списка равны нулю. После этого вставить в конец списка элемент, равный $-n$. Вывести полученный список.

Решение:

```
# Ввод
n = int(input('Введите целое число N '))

# Заполнение списка
arr = [0] * 100 # Заполним 100 нулями
min_n = 100

if n < min_n: # Чтобы не выйти за границы
    min_n = n

i = 0
while i < min_n:
    i += 1
    arr[i-1] = i

# Вставка -n в конец
arr.append(-n)

# Вывод
print(arr)

input()
```

Описание.

В начале задачи происходит ввод n . Метод *input* выводит строку-приглашение на ввод «Введите целое число N » и ждет ввода. Сам метод

возвращает строку, а не число, поэтому полученная строка преобразуется к целому числу методом приведения типов *int*. Результат преобразования записывается в переменную *p*.

Далее список *arr* заполняется 100 нулями с помощью операции повторения.

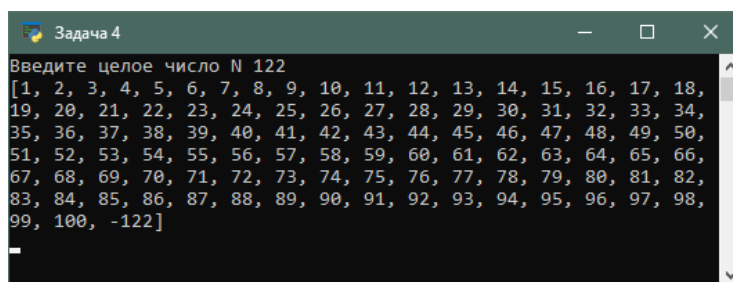
На данном этапе решения задачи возникает вопрос — что если пользователь введет число больше 100? Чтобы не выйти за границы списка, правильно будет обращаться к списку по индексам от 0 до минимума из *n* и 100. Этот минимум помещается в переменную *min_p*.

Дальше происходит вызов цикла *while*, который перезаписывает первые *min_p* нулей на нужные значения. Следует отметить, что если пользователь ввел $n < 0$, то и *min_p* будет меньше нуля. Следовательно, условие $i < min_p$ не выполнится ни разу и ошибки все равно не возникнет.

После этого остается только вставить $-n$ в конец списка с помощью функции *append*.

В конце задачи происходит вывод результата на консоль.

Результат.



```
Задача 4
Введите целое число N 122
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18,
19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34,
35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50,
51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66,
67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82,
83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98,
99, 100, -122]
```

9.2. Задачи на форматированный вывод

Все задачи должны быть выполнены с помощью оператора *%*, то есть без конкатенаций, повторений строк и циклов.

1. В коде даны шесть чисел с плавающей точкой. Создать строку формата, в которой эти числа будут располагаться в 3 столбца: в первом

и третьем столбцах выравнивание по правому краю, во втором — по левому. Все числа имеют 3 знака после запятой. Вывести данную строку.

2. В коде даны 2 строки и 3 целых числа. Создать строку формата, в которой будут чередоваться эти числа и строки. Сразу после чисел должна идти единица измерения — проценты. Вывести данную строку.

3. В коде даны шесть чисел с плавающей точкой. Создать строку формата, в которой эти числа будут располагаться в 2 столбца. Все числа имеют 5 знаков после запятой и выравнивание по левому краю, а также указан знак числа. Вывести данную строку.

4. В коде даны 2 числа и 2 строки с указанием единиц измерения. Создать строку формата, в которой эти числа будут представлены в степенной форме с указанием знака числа и точностью 2 знака после запятой. После чисел указываются их единицы измерения. Вывести данную строку.

5. В коде даны 6 строк. Создать строку формата, в которой эти строки будут расположены «лесенкой» Маяковского по 3. После первых двух строк в каждой «лесенке» добавить символ запятой, после последней — восклицательный знак. Вывести данную строку.

6. В коде даны 3 строки и 3 целых числа — номера страниц. Создать строку формата, в которой будет храниться оглавление: строка-название, затем отступ необходимой длины и номер страницы вида «стр. N». Вывести данную строку.

7. В коде даны 6 строк по 6 символов в каждой. Создать строку формата, в которой эти строки будут располагаться пирамидкой. Вывести данную строку.

8. В коде дано число с плавающей точкой x . Создать строку формата s_1 , в которой это число представлено как число со степенью. Создать строку s_2 , в которой это число представлено как число с плавающей точкой

и точностью 7 знаков. Затем создать строку формата s, в которой на первой строке находится s1 с выравниванием по правому краю, на следующей — s2 с выравниванием по левому. Ширина каждой строки в s равна 80 символов. Вывести s на консоль.

9. В коде даны 3 строчки: кому, от кого и текст заявления, а также дата в виде 3 целых чисел. Создать строку формата заявления и вывести ее на консоль.

10. В коде дана строка x из 9 символов, хранящая результат партии в крестики-нолики, например 'oxxxooxxo'. Создать строку формата, хранящую результат в виде поля 3 на 3 с применением псевдографики (). Вывести данную строку.

9.3. Задачи на списки

1. Пользователь вводит целое число n. Создать список из 50 элементов. Первые n значений — четные числа от 2 до 2*n. Остальные элементы списка равны нулю. После этого вставить в середину списка элемент, равный 10*n. Вывести полученный список.

2. Пользователь вводит число за числом, пока не введет 0. Все эти числа помещаются в конец изначально пустого списка. После ввода пользователем нуля весь список выводится на консоль. После этого вывести среднее арифметическое всех элементов списка.

3. Пользователь вводит строку. Строка разбивается и посимвольно записывается в список. Вывести список в прямом и обратном порядке.

4. Пользователь вводит 10 целых чисел. Поместить каждое из них в список a или список b в зависимости от их четности. Вывести оба списка с указанием, какие из чисел хранятся в конкретном списке.

5. Пользователь вводит 10 целых чисел. Сохранить их в список. Создать список из абсолютных значений этих чисел. Вывести на консоль сначала его, а затем исходный список.

6. Пользователь вводит 10 целых чисел. Сохранить их в список. Создать список из знаков этих чисел (-1, 0, 1). Вывести на консоль сначала его, а затем исходный список.

7. Пользователь поэлементно вводит матрицу 3*3 из целых чисел. Вывести матрицу на консоль, а затем число повторений самого первого элемента матрицы.

8. Пользователь вводит целое число n . Создать список из 100 элементов. Последние n значений — натуральные числа, уменьшающиеся от 100 до $100-n+1$. Остальные элементы списка равны нулю. После этого вставить в начало списка элемент, равный 100. Вывести полученный список.

9. Пользователь вводит целые числа n и k . Создать список из n натуральных чисел и вывести его на консоль. Затем удалить из него все числа кратные k и вывести результат.

10. Пользователь вводит целые числа n и k . Создать список из 100 натуральных чисел. Изменить этот список так чтобы после первых n чисел они повторялись еще раз, а затем шли все оставшиеся. Вывести этот список, а также все позиции (индексы), на которых встречается элемент k .

10. Модульность

10.1. Модули

Модуль — это функционально законченный фрагмент программы, который оформляется в виде отдельного файла.

Каждый файл исходного кода Python, чье имя заканчивается расширением `.py`, является модулем. Чтобы сделать файл модулем, никакого специального кода или синтаксиса не требуется: модулем будет любой файл такого рода. Другие файлы могут получать доступ к элементам,

определенным в модуле, импортируя этот модуль. Содержимое модуля делается доступным внешнему миру через его атрибуты.

Такая модель служб на основе модулей является основной идеей программной архитектуры в Python. Один из модулей предназначен быть главным файлом либо файлом верхнего уровня, или *сценарием* — файлом, запускаемым для старта целой программы, которая выполняется строка за строкой обычным образом. Ниже данного уровня все модули импортируют другие модули.

Импортируемый сценарий работает по умолчанию только однократно на сеанс. После первого импорта дальнейшие операции импорта того же модуля ничего не делают, поскольку это слишком затратная операция, чтобы повторять ее более одного раза в файле на выполнение программы.

Если на самом деле необходимо заставить Python выполнить файл снова в том же самом сеансе, не останавливая его и не перезапуская, то понадобится вызвать функцию `reload`, доступную в стандартном библиотечном модуле `imp`. Функция `reload` ожидает передачи имени объекта уже загруженного модуля, поэтому нужно иметь успешно импортированный модуль, прежде чем перезагружать его.

Перезагрузка не обладает свойством транзитивности — она воздействует только на указанный модуль, но не на модули, которые он может импортировать, поэтому иногда приходится перезагружать множество файлов.

Модуль в Python представляет собой пакет имен переменных, известный под названием *пространство имен*, и имена внутри такого пакета называются *атрибутами*. Атрибут — это просто имя переменной, которое прикрепляется к специфическому объекту (вроде модуля). Импортирующие модули получают доступ ко всем именам, определенным на верхнем уровне файла модуля. Имена обычно назначаются

инструментам, экспортируемым модулем (функциям, классам, переменным и т.д.), которые предназначены для использования в других файлах и программах. Снаружи имена файла модуля можно извлечь с помощью двух операторов: `import` и `from`, а также вызова `reload`.

Например, файл `mod_0.py` содержит единственную строку:

```
info = 'It is mod_0.py'
```

Файл `my_program.py` импортирует его, и использует переменную `info`:

```
import mod_0

print(mod_0.info)
```

Так `my_program.py` выведет строку «It is mod_0.py» — значение переменной `info`, установленное в модуле `mod_0.py`.

В общем случае синтаксис выражения с точкой *объект.атрибут* позволяет извлечь любой атрибут, прикрепленный к любому объекту, и является одной из самых распространенных операций в коде Python.

В качестве альтернативы извлекать (в действительности копировать) имена из модуля можно с помощью операторов `from`. Тогда файл `my_program.py` будет иметь следующий вид:

```
from mod_0 import info

print(info)
```

Таким образом, можно ссылаться на импортированную строку как на переменную `info`, а не как на ссылку на атрибут `mod_0.info`.

Следует обратить внимание, что в `import` и `from` имя модуля указывается в виде `mod_0` без расширения `.py`.

Для извлечения списка всех имен, доступных внутри модуля, можно использовать встроенную функцию `dir`.

10.2. Конфликты имен в модулях

Программы Python состоят из множества файлов модулей, связанных вместе операторами `import`, и каждый файл модуля является пакетом переменных, т.е. пространством имен. Не менее важно и то, что каждый модуль представляет собой автономное пространство имен: один файл модуля не может видеть имена, определенные в другом файле, до тех пор, пока явно не импортирует другой файл. Из-за этого модули содействуют минимизации конфликтов имен в коде — поскольку каждый файл является автономным пространством имен, имена в одном файле не могут конфликтовать с именами в другом файле, даже если они написаны одинаково.

Однако, при импортировании могут возникать конфликты, например, модули `math` и `cmath` имеют множество функций, которые являются общими для обоих из них, такие как `log10()`, `acos()`, `cos()`, `exp()`.

Если импортировать модули полностью, с помощью *from модуль import **, то модуль, импортированный позже, переопределяет совпадающие имена.

Более корректно будет импортировать только требуемые имена, например:

```
from math import cos, log2
```

Лучший вариант импорта — использование варианта *import модуль*. В таком случае все имена доступны только через имя модуля с точкой, и конфликты имен вообще не возникают.

11. Функции

11.1. Основные понятия

Функция (процедура, подпрограмма) является способом группирования набора операторов, позволяющим выполнять их более одного раза

в программе — упакованной процедурой, вызываемой по имени. Функции способны вычислять результирующее значение и также дают возможность указывать параметры, которые служат входными данными функции.

Функции представляют собой наиболее базовую программную структуру Python, которая предназначена для многократного использования кода.

Функции являются инструментом разбиения систем на составные элементы с хорошо определенными ролями. Они помогают разбить общую задачу на части — по одной функции для каждой подзадачи процесса. Гораздо проще реализовать небольшие подзадачи обособленно, чем полный процесс сразу.

Функции Python пишутся с помощью оператора *def*. В отличие от функций в компилируемых языках, таких как C, *def* представляет собой исполняемый оператор — пользовательская функция не существует до тех пор, пока Python не встретит и не выполнит *def*.

Когда Python достигает и выполняет оператор *def*, генерируется новый объект функции, который присваивается имени функции. Как и со всеми присваиваниями, имя функции становится ссылкой на объект функции.

Когда функция вызывается, вызывающий код приостанавливается до тех пор, пока функция не завершит свою работу и не возвратит управление обратно. Функции, которые вычисляют значение, посылают его вызывающему коду посредством оператора *return*; возвращенное значение становится результатом вызова функции. Оператор *return* без значения просто возвращает управление в вызывающий код и отправляет стандартный результат *None*.

Аргументы передаются в функции следующим образом: изменение имени аргумента внутри функции не приводит к изменению

соответствующего имени в вызывающем коде, но модификация на месте переданных изменяемых объектов может изменить объекты. Например:

```
def change_value(name):  
    name = 500  
  
x = 5  
y = [0] * 3  
  
change_value(x)    # не меняет x  
change_value(y)    # не меняет y
```

Это пример изменения имени аргумента, то есть присваивания переменной другого значения. Такие изменения не отражаются на внешнем коде, и, фактически, функция `change_value` не делает ничего.

```
def change_list(name):  
    name[0] = 100  
  
y = [0] * 3    # y = [0, 0, 0]  
  
change_list(y)    # y = [100, 0, 0]
```

Данный метод меняет список на месте, поэтому эти изменения отражаются в вызывающем коде.

В Python никогда не объявляются типы переменных, аргументов или возвращаемых значений,

Общий синтаксис оператора *def* следующий:

```
def имя(арг1, ... аргN):  
  
    операторы
```

Блок операторов называется телом функции. В строке заголовка указывается имя функции и список из нуля или более аргументов (параметров) функции.

Оператор *return* в Python может появляться где угодно в теле функции; по достижении он заканчивает вызов функции и посылает результат обратно вызывающему коду. Оператор *return* сам по себе не обязателен; если он отсутствует, то выход из функции происходит, когда поток управления достигает конца тела функции.

Оператор *def* создает функцию, но не вызывает ее. После выполнения *def* функцию можно вызывать (выполнять) в своей программе, добавляя к имени функции круглые скобки.

Поскольку *def* — оператор, он может появляться везде, где допускается оператор, даже внутри других операторов, например:

```
if условие:
    def func()...
else:
    def func()...
```

Данный код демонстрирует возможность выбора между альтернативными определениями функции. Оператор *def* во многом похож на оператор *=*. Он просто присваивает имя во время выполнения.

Как и все остальное в Python, функции — это просто объекты; они явно записываются в память во время выполнения программы.

```
def print_a():
    print('a')

z = print_a# присваивание объекта функции

z() # печатается 'a'

z() # печатается 'a' еще раз
```

11.2. Область видимости

В случае использования имени в программе Python создает, изменяет или ищет имя внутри того, что называется *пространством имен* — месте, где имена существуют. Место присваивания имени в исходном коде определяет пространство имен, в котором оно будет существовать, и отсюда его область видимости.

По умолчанию все имена, которым выполнено присваивание внутри функции являются *локальными* и ассоциируются с пространством имен данной функции и никаким другим. То есть имена, присвоенные внутри `def`, могут быть видны только в коде внутри этого оператора `def`. Ссылаться на такие имена извне функции нельзя. Имена, присвоенные внутри `def`, не конфликтуют с переменными за пределами `def`, даже если те же самые имена применяются где-то в другом месте:

```
x = 100; print(x) # 100
```

```
def func():
```

```
    x = 5; print(x)
```

```
func()          # 5
```

```
print(x)        # 100
```

Для вложенных функций справедливо:

- если переменная присваивается внутри `def`, то она будет локальной в этой функции;
- если переменная присваивается в объемлющем `def`, тогда она будет нелокальной в отношении вложенных функций;
- если переменная присваивается за пределами всех `def`, то она будет глобальной в целом файле.

Когда внутри функции указывается имя, Python ищет его в четырех местах — в локальной (`local`) области видимости, затем в локальных

областях видимости любых объемлющих (enclosing) функций, далее в глобальной (global) области видимости и, наконец, во встроенной (built-in) области видимости — и останавливает поиск на первом же месте, где обнаруживается имя.

Когда внутри функции выполняется присваивание имени, Python всегда создает либо изменяет имя в локальной области видимости, если только оно не было объявлено в этой функции как глобальное *global* или нелокальное *nonlocal*.

Когда производится присваивание имени за пределами всех функций, локальная область видимости совпадает с глобальной — пространством имен модуля.

В Python есть еще три области видимости — временные переменные циклов (во *включениях*), переменные ссылок на *исключения* в некоторых обработчиках *try* и локальные области видимости в операторах *class*.

Встроенная область видимости представляет собой всего лишь встроенный модуль под названием *builtins*, который содержит встроенные исключения, встроенные функции и имена *None*, *True* и *False*.

Оператор *global* дает возможность изменять имена, которые существуют вне *def* на верхнем уровне файла модуля.

Оператор *nonlocal* почти идентичен ему, но применяется к именам в локальной области видимости объемлющего оператора *def*, а не к именам во включающем модуле.

Оператор *global* состоит из ключевого слова *global*, за которым следует одно и более имен, разделенных запятыми. Присваивание или ссылка на все перечисленные имена в теле функции будет приводить к их отображению на область видимости включающего модуля, например:

```
cat = 'cat'

def dogging():
    global cat
    cat = 'dog'

dogging()

print(cat) # dog
```

12. Прочие основные типы

12.1. Файлы

Объекты файлов являются главным интерфейсом к внешним файлам на компьютере. Они могут применяться для чтения и записи текстовых заметок, аудиоклипов, документов Excel, сохраненных сообщений электронной почты и многого другого.

Специфический литеральный синтаксис для создания объекта файлов отсутствует. Взамен, чтобы создать объект файла, необходимо вызвать встроенную функцию *open*, передав ей в виде строк имя внешнего файла и необязательный режим обработки. Она создает файловый объект Python, который служит ссылкой на файл, находящийся на компьютере.

Простейший пример работы с файлом:

```
f = open('text.txt', 'w')
f.write('Hello there\n')
f.close()
```

В первой строке кода был открыт на запись (указан режим обработки 'w') файл `text.txt`. Далее в него была записана строка «Hello there\n», после чего файл был закрыт для сбрасывания буферов вывода на диск.

Если в вызове строка режима не указана, файл открывается в режиме обработки 'r' для чтения текстового ввода.

Существует семь режимов открытия:

- 'r' — открытие на чтение;
- 'w' — открытие на запись, содержимое файла удаляется, если файла не существует, создается новый;
- 'x' — открытие на запись, если файла не существует, иначе исключение;
- 'a' — открытие на дозапись, информация добавляется в конец файла;
- 'b' — открытие в двоичном режиме;
- 't' — открытие в текстовом режиме;
- '+' — открытие на чтение и запись.

Функция *read* читает целый файл (или указанное число символов) в одиночную строку. Функция *readline* читает следующую строку файла, а функция *readlines* читает целый файл в список строк.

Лучший способ чтения файла в наше время — вообще его не читать, поскольку файлы предлагают итератор, который автоматически производит чтение строка за строкой в циклах *for* и других контекстах:

```
for line in open('text.txt'): print(line)
```

Пример выводит все строки файла *text.txt*.

Для записи используются функции *write* и *writelines*. Функция *close* вручную закрывает файл, но это делает автоматически сборщик мусора.

Данные, прочитанные из файла, всегда возвращаются в виде строки, поэтому ее необходимо преобразовывать в объект Python другого типа явно. Аналогично Python не добавляет форматирование и не преобразует объекты в строки при записи — требуется предоставлять уже сформатированные строки. По этой причине при работе с файлами будут полезными инструменты, предназначенные для преобразования объектов

в и из строк (например, *int*, *float*, *str* и выражение и метод форматирования строк).

По умолчанию выходные файлы всегда буферизируются, а это значит, что записываемый текст может не сразу быть передан из памяти на диск — сбрасывание буферизированных данных на диск инициирует закрытие файла или вызов метода *flush*.

Чтобы получить доступ к файлам, содержащим отличающийся от ASCII текст Unicode представленного ранее вида, можно передавать имя кодировки, если она не совпадает со стандартной кодировкой для имеющейся платформы:

```
f = open('text.txt', 'w', encoding='utf-8')
```

Текстовые файлы представляют содержимое в виде нормальных строк *str*, автоматически выполняя кодирование и декодирование Unicode при записывании и чтении данных, тогда как *двоичные файлы* представляют содержимое в виде специальных строк *bytes* и позволяют получать доступ к содержимому файла, не изменяя его.

Для проверки наличия файла можно использовать функцию *exists* модуля *os.path*:

```
import os.path

filename = 'somefile.txt'

if os.path.exists(filename):
    print('Y')
else:
    print('N')
```

12.2. Множества

Множества не являются ни отображениями, ни последовательностями. Это неупорядоченные коллекции уникальных и неизменяемых объектов, которые поддерживают операции, соответствующие математической теории множеств. По определению, элемент встречается во множестве только однажды независимо от того, сколько раз он добавлялся.

Множества создаются путем вызова встроенной функции `set` либо использования литералов и выражений множеств и поддерживают обычные математические операции над множествами:

```
s1 = set('abc')
```

```
s2 = {'a', 'b', 'c'}
```

Следующий пример показывает применение операций разности, объединения, пересечения, симметрической разности и проверки на надмножество:

```
x = set('abcd')
```

```
y = set('cdefg')
```

```
x - y # {'b', 'a'}
```

```
x | y # {'e', 'c', 'd', 'b', 'g', 'a', 'f'}
```

```
x & y # {'d', 'c'}
```

```
x ^ y # {'b', 'e', 'g', 'a', 'f'}
```

```
x > y # False
```

Множества являются удобными для решения общих задач, таких как фильтрация дубликатов, нахождение разностей и выполнение нейтральных к порядку проверок равенства без сортировки в списках, строках и всех остальных итерируемых объектах.

Множества также поддерживают операцию проверки членства `in`.

В дополнение к выражениям, объект множества предоставляет методы, которые соответствуют этим и другим операциям и поддерживают изменения множеств: метод *add* вставляет один элемент, *update* представляет собой объединение на месте, а *remove* удаляет элемент по значению.

Как итерируемые контейнеры, множества могут также использоваться в операциях вроде *len*, циклах *for* и списковых включениях. Тем не менее, из-за того, что множества не упорядочены, они не поддерживают операции над последовательностями, подобные индексации и нарезанию.

Множества являются мощными и гибкими объектами, но имеют одно ограничение — из-за своей реализации множества способны содержать только объекты неизменяемых типов. Следовательно, списки и словари не могут встраиваться во множества, но кортежи могут, если необходимо хранить составные значения.

12.3. None

Объект *None* представляет собой единственное значение специального типа данных в Python и обычно служит пустым заполнителем (во многом похоже на указатель *NULL* в C).

Используется, например, чтобы заранее выделить память под 100-элементный список и предоставить возможность работы с элементами по любому из 100 смещений, можно заполнить его объектами *None*:

```
ar = [None] * 100
```

Следует иметь в виду, что *None* не означает «неопределенный». То есть *None* представляет собой что-то, а не ничего — это реальный объект и реальный участок памяти, который создается и назначается встроенному имени самим Python.

12.4. Объект `type`

В Python даже сами типы представляют собой объекты: тип объекта является объектом типа `type`.

Встроенная функция `type` возвращает тип другого объекта.

Наглядное объяснение этого продемонстрировано в следующем примере:

```
s = set('abc')  
  
print(type(s))
```

Данный код выведет `<class 'set'>` — это означает, что типом `s` является `set` то есть множество.

На практике объект `type` позволяет коду проверять типы обрабатываемых объектов:

```
if type(s) == set:  
    print('It is set')
```

13. Задачи

13.1. Пример решения

Задача 5. Написать консольное приложение с меню, позволяющее:

- считать словарь из текстового файла;
- напечатать словарь на консоль;
- сохранить словарь в текстовый файл;
- добавить значение в словарь;
- поиск по ключу;
- вывести объем словаря.

Реализовать алфавитный указатель, хранящий номер страницы по ключевому слову.

Решение:

```
choise = -1 # Выбор пункта меню пользователем

dct = {}

file_name = r'program_5_dict.txt'

while choise: # Пока не "пункт 0 - выход"

    # Печать меню

    print('Выберите действие')

    print('[1] Вывести текущий словарь')

    print('[2] Прочитать из файла')

    print('[3] Сохранить в файл')

    print('[4] Добавить значение')

    print('[5] Найти по ключу')

    print('[6] Вывести объем словаря')

    print('[0] Завершить программу')

    # Ввод

    s_choise = input()

    if not s_choise.isdigit():

        print('-- Повторите попытку ввода --\n')

        continue

    choise = int(s_choise)
```

```

# Выбор

if choise==0: # Завершить

    ...

elif choise==1: # Вывести

    if dct:

        to_print = dct.items()

        for i in to_print:

            print('%-40s %d' % (i[0], i[1]))

    else:

        print('-- Словарь пуст --')

elif choise==2: # Прочитать

    if dct:

        dct.clear()

    f = open(file_name, 'rt')

    key = True

    val = True

    i = 0

    while key and val:

        key = f.readline()

        val = f.readline()

        if key and val: # Успешное чтение

            key = key[:-1]

```

```

        val = int(val[:-1])

        dct[key] = val # Добавляем

        i += 1

    f.close()

    print('-- Считано %d значений --' % i)
elif choise==3: # Сохранить
    f = open(file_name, 'wt')
    to_write = dct.items()
    for i in to_write:
        f.write(i[0]+'\\n'+str(i[1])+'\\n')
    f.close()
    print('-- Файл перезаписан --')
elif choise==4: # Добавить
    txt = input('Введите строку: ')
    num = int(input('Введите № страницы: '))
    dct[txt] = num
    print('-- Значение добавлено --')
elif choise==5: # Найти
    txt = input('Введите строку: ')
    res = dct.get(txt)
    if res:
        print('Страница %d' % res)

```

```

else:

    print('-- Значение не найдено --')

elif choise==6: # Длина

    to_list = dct.keys()

    print('Объем словаря - %d значений'

          % len(to_list))

else: # Иначе

    print('-- Выбран недопустимый пункт --')

print()

```

Описание.

Данная задача кажется объемной, но не должна вызывать трудностей при рассмотрении ее по шагам.

В начале программы задаются три переменные: `choise`, которая служит для реализации консольного меню, `dct` — текущий хранимый в памяти приложения словарь, который пуст до тех пор, пока в него не считаются данные с консоли или из файла и `file_name` — переменная с именем файла.

Далее начинается цикл. Он служит для того, чтобы пункты меню повторялись до тех пор, пока пользователь не выберет пункт «выход».

Первые 9 строк цикла — просто печать меню по строкам. Следующие 6 строк — блок ввода. Сначала пользовательское значение сохраняется как строка в переменную `s_choise`, а затем, с помощью функции `isdigit`, проверяется, является ли эта строка числом во избежание ошибок. Если пользователь ввел недопустимое значение, то сообщение об этом выводится на консоль, и программа переходит в начало цикла с помощью оператора `continue`, меню печатается снова, и снова ожидается ввод.

Как только ввод прошел успешно, начинается сложный *if*. Первым его пунктом происходит проверка на выход из меню `choise==0`. Если да — управление передается за все ветви оператора *if* с помощью оператора многоточия. После этого цикл начинается званого с проверки на ноль, и задача завершается. Подобных же результатов можно было добиться с помощью операторов *break* или *continue*.

Следующая ветвь цикла — вывод словаря. Чтобы напечатать словарь, достаточно было вызвать метод `print(dct)`. Но для удобства чтения словарь сначала преобразуется к списку кортежей с помощью функции *items*, и затем печатается с помощью цикла *for*.

Первая половина работы с файлом в данной задаче — это его чтение. Для этого сначала текущий словарь очищается, а файл открывается на чтение. Данные в файле хранятся так: на одной строке ключ, на следующей — значение, далее следующий ключ и следующее значение — каждое на своей строке. Поэтому необходимо читать сразу пару строк, для чего был реализован цикл, использующий функции *readline*. После цикла файл закрывается.

Следующая возможность — сохранить файл. Файл открывается в режиме «перезаписи» *w*, то есть данные заменяются. Словарь преобразуется к списку и записывается так, что каждый ключ и каждое значение помещается в свою строку файла с помощью функции *write*. После записи файл закрывается.

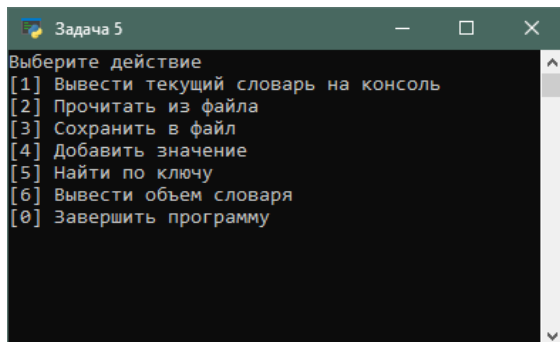
Реализация добавления в словарь предельно проста. Вводится ключ, вводится значение, значение присваивается в словарь по ключу. Если ключа не было, он создается, если был — значение в словаре обновляется.

Поиск осуществляется с помощью метода *get*, который возвращает значение по ключу, либо `None`, если значение не найдено.

Объем словаря находится путем преобразования его к списку, чья длина вычисляется функцией *len*.

Пункт «иначе» возможен только когда пользователь ввел число, но это число не обрабатывается ни *if* ни одним *elif*, то есть число меньше 0 или больше 6.

Результат.



Задача 6. Выполнить предыдущую задачу, разбив ее на функции.

Решение:

```
choise = -1 # Выбор пункта меню пользователем
```

```
dct = {}
```

```
file_name = r'program_5_dict.txt'
```

```
#Вывод на консоль
```

```
def prinr_dict(dict_name):
```

```
    if type(dict_name) is dict:
```

```
        if dict_name:
```

```
            to_print = dict_name.items()
```

```
            for i in to_print:
```

```
                print('%-40s %d' % (i[0], i[1]))
```

```

        else:
            print('-- Словарь пуст --')
    else:
        print('-- Invalid Argument --')
# Объем словаря
def dict_size(dict_name):
    to_list = dict_name.keys()
    return len(to_list)
# Часть кода пропущена
while choise: # Пока не "пункт 0 - выход"
    print_menu() # Печать меню
    choise = input_int() # Ввод
    # Выбор
    if choise==0: # Завершить
        ...
    elif choise==1: # Вывести
        printr_dict(dct)
    elif choise==2: # Прочитать
        dct = file_to_dict(file_name)
    elif choise==3: # Сохранить
        dict_to_file(file_name, dct)

```

```

elif choise==4: # Добавить

    txt = input('Введите строку: ')

    num = int(input('Введите № страницы: '))

    dct[txt] = num

    print('-- Значение добавлено --')

elif choise==5: # Найти

    txt = input('Введите строку: ')

    res = dct.get(txt)

    if res:

        print('Страница %d' % res)

    else:

        print('-- Значение не найдено --')

elif choise==6: # Длина

    print('Объем словаря - %d значений'

          % dict_size(dct))

else: # Иначе

    print('-- Выбран недопустимый пункт --')

print()

```

Описание.

В начале файла определены все используемые функции.

Первая функция — `print_dict` — функция печати словаря на консоль. Она принимает словарь как аргумент и не возвращает какого-либо значения. Главное отличие этой функции от решения предыдущей задачи — это проверка типа передаваемого аргумента `type(dict_name) is`

`dict`. Она может быть необходима, поскольку невозможно явно ограничивать тип аргументов функции.

Другой момент, на который следует обратить внимание, это сам факт передачи аргумента. Переменная словаря `dict` объявляется в самом модуле и поэтому становится глобальной по отношению к функциям того же модуля. Из этого следует, что возможно было реализовать функцию без параметров, которая бы обращалась напрямую к `dict`, а не к параметру `dict_name`. Однако в таком случае данная функция не могла бы быть применима к словарям в общем, а только к конкретному словарю `dict`. Это означает, например, то, что вызов в другом модуле для собственного словаря этой функции не имел бы смысла (или в этом же модуле для другого словаря).

Вторая описанная функция `dict_size` — это пример функции, возвращающей значение с помощью ключевого слова *return*.

Реализация некоторых функций опущена для краткости, после чего показано само приложение, полностью аналогичное предыдущей задаче. Оно несколько более структурировано за счет вызова функций. Также функции позволили бы избежать дублирования кода, например, если бы был реализован ввод с проверкой целочисленных значений не только в меню, но и при добавлении элементов в словарь.

13.2. Список задач

Написать консольное приложение с меню, позволяющее:

- считать словарь из текстового файла;
- напечатать словарь на консоль;
- сохранить словарь в текстовый файл;
- добавить значение в словарь;
- поиск по ключу;
- +1 возможность из задачи.

1. Реализовать телефонную книгу, хранящую ФИО по номеру телефона. Добавить возможность удалять запись из словаря.

2. Реализовать магазин, хранящий название и цену товара по артикулу. Добавить возможность напечатать только названия всех хранимых товаров.

3. Реализовать базу ГАИ, хранящую марку машины по автомобильному номеру. Добавить возможность удалять запись из словаря.

4. Реализовать список Форбс, хранящий состояние человека по его фамилии. Добавить возможность выводить топ-3 словаря по состоянию.

5. Реализовать базу тюрьмы, хранящую номер статьи заключенного по его фамилии. Добавить возможность печати количества заключенных.

6. Реализовать Кинопоиск, хранящий режиссера по названию фильма. Добавить возможность поиска фильмов по режиссеру.

7. Реализовать алфавитный указатель, хранящий номер страницы по ключевому слову. Добавить возможность удалять запись из словаря.

8. Реализовать календарь, хранящий название праздника по дате. Добавить возможность сохранять в заданный пользователем файл.

9. Реализовать таблицу Брадиса, хранящую значение синуса и косинуса по углу. Добавить возможность вычисления тангенса и арктангенса для угла.

10. Реализовать азбуку Морзе, хранящую код по символу. Добавить возможность переводить все известные символы из введенной строки.

Список литературы

1. Лутц М. Изучаем Python. Т. 1, 5-е изд. – ООО «Диалектика», 2019.
2. Комментирование Python кода – URL: <https://dev-gang.ru/article/kommentirovanie-python-koda-auf6lgv2vv/> (дата обращения: 01.03.2021).
3. Модуль decimal – URL: <https://metanit.com/python/tutorial/6.4.php> (дата обращения: 01.03.2021).
4. Что такое пространства имен Python – URL: <https://code.tutsplus.com/ru/tutorials/what-are-python-namespaces-and-why-are-they-needed--cms-28598> (дата обращения: 01.03.2021).
5. Цикл for – URL: https://pythontutor.ru/lessons/for_loop/ (дата обращения: 01.03.2021).

У ч е б н о е и з д а н и е

ЯЗЫК ПРОГРАММИРОВАНИЯ PYTHON

Часть 1

Учебно-методическое пособие

Составители:

**Артемов Михаил Анатольевич,
Барановский Евгений Сергеевич,
Меджидов Руслан Гусенович**

Издано в авторской редакции

Подписано в печать 14.05.2021. Формат 60×84/16
Усл. п. л. 5,4. Тираж 20 экз. Заказ 175

Издательский дом ВГУ
394018 Воронеж, пл. Ленина, 10
Отпечатано с готового оригинал-макета
в типографии Издательского дома ВГУ
394018 Воронеж, ул. Пушкинская, 3